Intermediate Code Generation using Syntax-Directed Definitions (SDD) and Syntax-Directed Translation (SDT)

Lecture Notes based on *Compilers: Principles, Techniques, and Tools* (the "Dragon Book"), 2nd Edition, Aho, Lam, Sethi, Ullman – Chapters 5 and 6

Suggested Lecture Plan (3–4 lectures)

Lecture 1 Role of intermediate code; forms of IR: syntax trees, DAGs, three-address code, quadruples and triples (indirect triples as a refinement), SSA form and the φ-function.

Lecture 2 *Syntax-directed definitions and translation, from scratch:* why syntax alone is not enough, attributes, semantic rules, synthesized and inherited attributes, dependency graphs, S- and L-attributed definitions; then SDTs: actions inside productions, postfix translation, marker nonterminals.

Lecture 3 Translation of expressions and assignments, array references, type checking and conversion.

Lecture 4 Boolean expressions and short-circuit code, flow-of-control statements, back-patching, switch statements and procedure calls; revision and exercises.

Contents

1 Introduction: Why Intermediate Code? 4

1.1 Advantages of using an intermediate representation 4

2 Variants of Syntax Trees and Intermediate Representations 4

2.1 Syntax trees (abstract syntax trees) 4

2.2 Directed acyclic graphs (DAGs) for expressions 5

2.2.1 The value-number method for constructing DAGs 6

2.3 Three-address code (TAC) 9

2.3.1 Generating three-address code from a DAG 9

2.3.2 Addresses and instructions 10

2.3.3 Representations of three-address code 10

2.3.4 Static single-assignment (SSA) form and the φ-function 12

3 Syntax-Directed Definitions (SDD) 15

3.1 What is an SDD, and what is it for? . . . 15
3.2 Attributes . . . 17
3.3 Semantic rules: a first tiny example . . . 18
3.4 Formal definition . . . 19
3.5 Synthesized and inherited attributes . . . 19
3.6 Annotated parse trees and dependency graphs . . . 20
3.7 Classes of SDDs that are easy to implement . . . 22
3.8 More examples: L-attributed and general (non-circular) SDDs . . . 23
3.9 SDD for constructing syntax trees . . . 25
3.10 SDD for constructing DAGs directly . . . 26

4 Syntax-Directed Translation Schemes (SDT) 28
4.1 From specification to implementation . . . 28
4.2 Definition and the meaning of an action's position . . . 29
4.3 Postfix translation schemes . . . 30
4.4 SDTs with actions inside productions . . . 30
4.5 Marker nonterminals (actions in the middle, for bottom-up parsing) . . . 32
4.6 Eliminating left recursion from an SDT . . . 33
4.7 Converting an L-attributed SDD into an SDT . . . 34

5 Translation of Expressions and Assignments 35
5.1 SDD for three-address code . . . 35
5.2 Incremental translation (SDT with `gen` emitting directly) . . . 36
5.3 Addressing array elements . . . 37
5.4 Translation of array references . . . 38

6 Type Checking and Type Conversion 39
6.1 Type conversion (coercion) . . . 40

7 Control Flow: Boolean Expressions and Statements 41
7.1 Short-circuit (jumping) code . . . 41
7.2 Flow-of-control statements: SDD with inherited labels . . . 42
7.3 SDD for boolean expressions (jumping code) . . . 44
7.4 Avoiding redundant gotos (fall-through) . . . 45

8 Backpatching 46
8.1 The problem and the idea . . . 46
8.2 Boolean expressions . . . 46
8.3 Flow-of-control statements . . . 47

8.4 Comparison: inherited labels vs. backpatching 49

9 Switch Statements and Procedure Calls (Brief) 49

9.1 Switch statement 49

9.2 Procedure calls 50

10 A Complete Worked Example 51

11 Summary and Key Takeaways 52

12 Practice Problems 53

1 Introduction: Why Intermediate Code?

In the *analysis–synthesis* model of a compiler, the **front end** analyses the source program and produces an **intermediate representation (IR)**; the **back end** synthesises the target program from that IR.

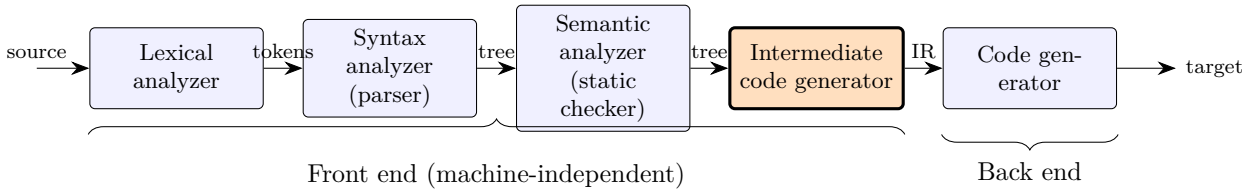


Figure 1: Position of the intermediate code generator in a compiler. The shaded (thick) box is the topic of these notes. In practice, static checking and intermediate code generation are often done together in one pass.

1.1 Advantages of using an intermediate representation

- **Retargeting.** A compiler for m languages and n machines needs only m front ends plus n back ends ($m+n$ pieces instead of $m \times n$ separate compilers) if they share one IR. For example, 3 languages and 4 machines need $3 + 4 = 7$ pieces instead of 12 compilers.
- **Machine-independent optimisation.** Optimisations (common sub-expression elimination, constant folding, loop-invariant code motion, ...) are applied once on the IR.
- **Simplicity.** Complex source constructs are lowered to a few simple operations, making code generation easier.

Static checking vs. intermediate code

Static checking includes type checking and syntactic checks that the grammar cannot enforce (e.g., `break` must be inside a loop). Dragon Book Chapter 6 treats type checking and intermediate-code generation together, because both are done by attaching attributes and actions to the grammar.

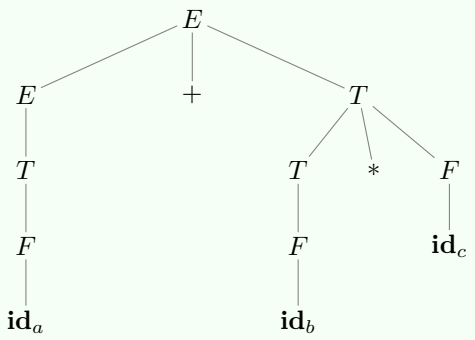
2 Variants of Syntax Trees and Intermediate Representations

2.1 Syntax trees (abstract syntax trees)

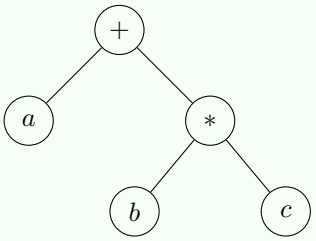
A **syntax tree** is a condensed form of the parse tree: operators and keywords appear as *interior nodes*, and operands as children. Chains of single productions (such as $E \to T \to F \to \mathbf{id}$) disappear, and so do punctuation tokens such as parentheses. The tree keeps only what later phases need: *which operation is applied to which operands.*

Example: parse tree vs. syntax tree for $a + b * c$

Take the usual grammar $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to \mathbf{id}$.



(a) parse tree: 13 nodes



(b) syntax tree: 5 nodes

Figure 2: Parse tree and syntax tree of $a + b * c$. Precedence survives in the shape of the syntax tree: $*$ is below $+$, so it is evaluated first.

Most parse-tree nodes only record *how the grammar was written* (the chains $E \to T \to F$); the syntax tree has 5 nodes. Parentheses never appear in a syntax tree: $(a + b) * c$ simply has the $+$ node below the $*$ node.

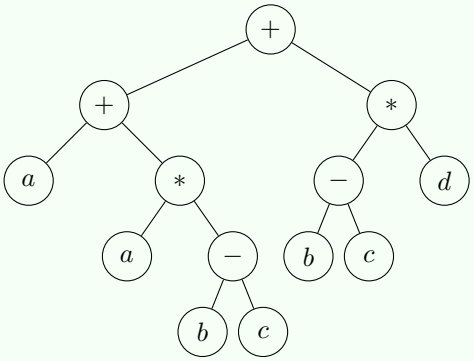
2.2 Directed acyclic graphs (DAGs) for expressions

A **DAG** identifies the *common sub-expressions*: a node is shared if the same sub-expression appears more than once. It shows the same information as a syntax tree but is more compact, and it directly suggests how to generate efficient code that computes a common sub-expression only once.

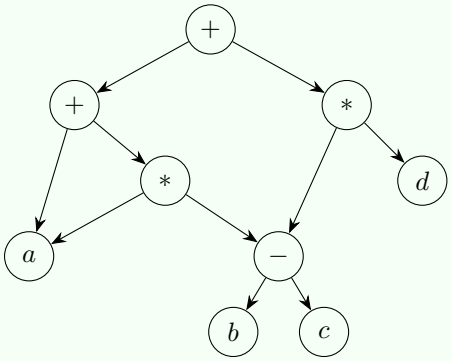
Example: tree vs. DAG for $a + a * (b - c) + (b - c) * d$

Figure 3 shows the syntax tree (left) and the DAG (right). In the DAG, the leaf a and the sub-expression $b - c$ are each shared by two parents.

Why arrows? A DAG is a *directed* acyclic graph. Each edge points from an operator node to one of its operands ("this operation uses that value"), so a node with two incoming arrows is a value that is used twice. There is no cycle because an expression can never be an operand of itself. In a tree the direction is the same but is left implicit, since every node has exactly one parent.



(a) Syntax tree



(b) DAG

Figure 3: Syntax tree and DAG for $a + a * (b - c) + (b - c) * d$. In the DAG every edge is an arrow from an operator to one of its operands.

2.2.1 The value-number method for constructing DAGs

The problem. While building the tree for an expression, we want to *reuse* a node if the same sub-expression has already been built. To do this we need a fast way to answer: "Have I already created a node for $b - c$?" The **value-number method** gives a simple answer. Here we assume the *syntax tree* of the expression has already been built (Figure 3(a)). We convert it into a DAG by visiting the tree nodes in **post-order** (left subtree, right subtree, then the node itself), so that the children of a node are always handled before the node.

The idea. Keep all nodes in an **array of records**. Each record describes one node of the DAG, and the *position (index) of the record in the array is the node's value number*. The value number is therefore just the node's *name*: "node 4" means "the record at position 4".

What the numbers mean

- **Value number** = the index of a record in the array = the identity of one DAG node. It is *not* the numeric value of the expression (the number 4 does not mean the expression evaluates to 4).
- A **leaf record** stores the kind of leaf and what it is: (**id**, pointer to symbol-table entry) for a variable, or (**num**, value) for a constant.
- An **interior record** has the form (op, *left*, *right*). Here op is the operator, and ***left* and *right* are the value numbers of the two child nodes**, i.e., they are *pointers* to other records (the operands).

Example. The record $(*,\ 1,\ 4)$ means: "a node for the operator $*$ whose left child is node 1 and whose right child is node 4". Looking up the array, node 1 is the leaf a and node 4 is $b - c$; so this record stands for $a * (b - c)$.

Algorithm: finding or creating a node

To create a node with signature $\langle op,\ l,\ r\rangle$ (where l and r are value numbers of the children that were already created):

1. **Search** the array for a record with exactly the same op, *left* $= l$, and *right* $= r$.
2. If such a record **exists**, do not create anything; **return its value number**. (This is the common sub-expression being shared.)
3. Otherwise **append** a new record $(op,\ l,\ r)$ at the end of the array and **return its index** as the new value number.

(For speed, a hash table keyed on (op, l, r) replaces the linear search in step 1.)

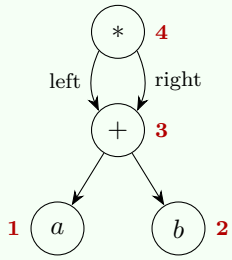
Why does comparing only (op, l, r) work? Two nodes represent the same expression exactly when they have the same operator and the same children. Children are the same exactly when they have the same value numbers (because every distinct sub-expression gets exactly one number). So, by building from the leaves upward, equal sub-expressions always get equal numbers.

A second example: $(a + b) * (a + b)$ and the commutativity trap

Post-order requests and the records they create:

1. leaf $a \Rightarrow$ new record 1; leaf $b \Rightarrow$ new record 2; $\langle +, 1, 2\rangle \Rightarrow$ new record 3.
2. Second copy: leaf $a \Rightarrow$ found (1); leaf $b \Rightarrow$ found (2); $\langle +, 1, 2\rangle \Rightarrow$ **found (3)**, shared.
3. $\langle *, 3, 3\rangle \Rightarrow$ new record 4 (both children are the same node).

The DAG has 4 nodes (the tree has 7) and gives `t1 = a + b`, `t2 = t1 * t1`. Note that the $*$ node has *two* arrows into the same node: its left and its right operand are one and the same value.



Now try $(a+b)*(b+a)$: the second sum is $\langle +, 2, 1\rangle$, which is *not* equal to $\langle +, 1, 2\rangle$, so a new node is created and $a+b$ is computed twice. To avoid this, a compiler stores the children of a commutative operator in a canonical order (say, smaller value number first); then $\langle +, 2, 1\rangle$ becomes $\langle +, 1, 2\rangle$ and is shared.

Limits of the method

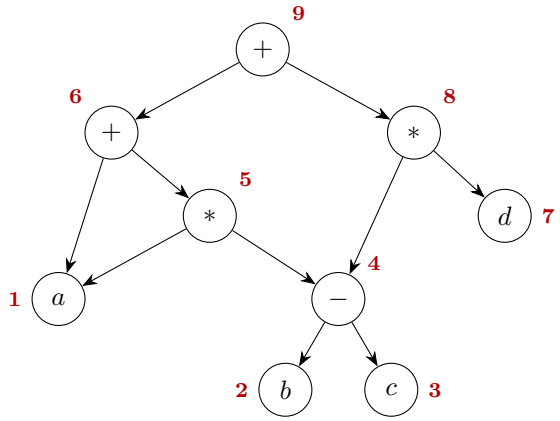
The test "same *op* and same children" finds only *syntactically identical* sub-expressions (up to the operand-order trick above). It does not know that $x*2$ and $x+x$ are equal; that needs algebraic simplification, a separate optimisation.

Step-by-step example: $a + a*(b-c) + (b-c)*d$**.** We walk the syntax tree of Figure 3(a) in post-order: a, a, b, c, then $-$, $*$, $+$ of the left part, then b, c, $-$, d, $*$, and finally the root $+$. At every tree node we ask the array for the matching DAG node. Table 1 shows each request and what the algorithm does.

Step	Request	Found in array?	Action	Returns
1	leaf a	no	create record 1: $(\mathbf{id}, a)$	1
2	leaf a (2nd time)	**yes, record 1**	nothing created	1
3	leaf b	no	create record 2: $(\mathbf{id}, b)$	2
4	leaf c	no	create record 3: $(\mathbf{id}, c)$	3
5	$\langle -, 2, 3\rangle$ $(b-c)$	no	create record 4: $(-, 2, 3)$	4
6	$\langle *, 1, 4\rangle$ $(a*(b-c))$	no	create record 5: $(*, 1, 4)$	5
7	$\langle +, 1, 5\rangle$ $(a+a*(b-c))$	no	create record 6: $(+, 1, 5)$	6
8	leaf b (2nd time)	**yes, record 2**	nothing created	2
9	leaf c (2nd time)	**yes, record 3**	nothing created	3
10	$\langle -, 2, 3\rangle$ ($b-c$ again)	**yes, record 4**	nothing created – *shared!*	4
11	leaf d	no	create record 7: $(\mathbf{id}, d)$	7
12	$\langle *, 4, 7\rangle$ $((b-c)*d)$	no	create record 8: $(*, 4, 7)$	8
13	$\langle +, 6, 8\rangle$ (whole expression)	no	create record 9: $(+, 6, 8)$	9

Table 1: Trace of the value-number method. Steps 2, 8, 9, 10 reuse existing nodes; step 10 is where the common sub-expression $b-c$ is detected.

The result has only 9 nodes (a plain syntax tree would have 13: it repeats a, b, c and the whole $b-c$ sub-tree). The final value number 9 identifies the root of the DAG.



red number = value number (index of the node's record in the array)

Arrows point from an operator to its operands.

Nodes 1 and 4 each have **two parents** (two incoming arrows): they are shared.

Figure 4: The DAG for $a + a * (b - c) + (b - c) * d$ with the value number of every node in red. Arrows go from operator to operand.

value no.	op	left	right
1	id a	–	–
2	id b	–	–
3	id c	–	–
4	−	2	3
5	$*$	1	4
6	+	1	5
7	id d	–	–
8	$*$	4	7
9	+	6	8

left = 1 points to record 1 (the leaf a)

right = 4 points to record 4 (the node $b - c$)

So record 5 = $(*, 1, 4)$ represents $a * (b - c)$.

Figure 5: The array of records after processing the example. Interior records contain *value numbers of their children*; here record 5 refers to records 1 and 4.

The whole procedure. Converting a syntax tree into a DAG is a single post-order traversal:

```
int buildDAG(TreeNode t) {            // returns a value number
    if (t is a leaf)
        return findOrCreateLeaf(t);   // search (id/num, entry); create if absent
    l = buildDAG(t.left);             // children first (post-order)
    r = buildDAG(t.right);
    return findOrCreateNode(t.op, l, r);   // search (op,l,r); create if absent
}
```

The "search" is only a *lookup of an identical record* in the array (or hash table); it is not a second traversal of the tree.

Looking ahead

Building the tree first and then converting it is easy to understand. Later, after we study syntax-directed definitions (Section 3.10), we will see that the same DAG can be built *directly while parsing*, without constructing the tree at all: the parser simply calls the find-or-create routine each time it recognises an operator.

2.3 Three-address code (TAC)

In **three-address code** every instruction has *at most one operator on the right side* and at most three addresses (two operands and one result). Compound expressions are broken into temporaries, so the order of evaluation is made explicit.

For $a + a * (b - c) + (b - c) * d$ the TAC (from the DAG, reusing $t_1 = b - c$) is:

```
t1 = b - c
t2 = a * t1
t3 = a + t2
t4 = t1 * d
t5 = t3 + t4
```

Without the DAG (from the tree) $b - c$ would be computed twice.

2.3.1 Generating three-address code from a DAG

Three-address code can be generated from the DAG alone; no SDD or SDT is needed.

Idea. Every *interior* node of the DAG becomes exactly *one* three-address instruction, with a fresh temporary holding its result. *Leaves* generate no instruction: they only supply a name (a variable such as a, or a constant). A shared node is still one node, so it produces only one instruction; this is how the DAG removes the common sub-expression.

Order. Process the records of the array in increasing order of value number. A node's children were always created before the node, so their value numbers are smaller, and their results are ready when the node is reached. The array order is therefore already a valid evaluation order.

```
for i = 1 to n:                          // records in the array, in index order
    if record i is a leaf:
        name[i] = the variable or constant it stands for
    else:                                // record i = (op, left, right)
        name[i] = newTemp()
        emit( name[i] = name[left] op name[right] )
```

Here $name[i]$ is the address that holds the value of node i. For an assignment $x = \langle\text{expression}\rangle$ we finish with one more instruction, `x = name[root]`.

Record	Content	$name[left]$	$name[right]$	$name[i]$	Instruction emitted
1	`id` a	–	–	a	(none, leaf)
2	`id` b	–	–	b	(none, leaf)
3	`id` c	–	–	c	(none, leaf)
4	$(-, 2, 3)$	b	c	t_1	`t1 = b - c`
5	$(*, 1, 4)$	a	t_1	t_2	`t2 = a * t1`
6	$(+, 1, 5)$	a	t_2	t_3	`t3 = a + t2`
7	`id` d	–	–	d	(none, leaf)
8	$(*, 4, 7)$	t_1	d	t_4	`t4 = t1 * d`
9	$(+, 6, 8)$	t_3	t_4	t_5	`t5 = t3 + t4`

Table 2: Generating three-address code from the array of Figure 5 (the DAG of $a + a * (b - c) + (b - c) * d$). Node 4 has two parents (nodes 5 and 8) but produces only one instruction; both parents use t_1.

Three ways to obtain three-address code.

Method	How	Common sub-expressions
1. From a finished DAG	Scan the array in index order (above); no SDD/SDT needed.	Computed once (shared nodes).
2. During DAG construction	Whenever find-or-create *creates* a node, emit its instruction; when it finds an existing node, emit nothing. Needs SDD/SDT (Section 3.10).	Computed once.
3. SDD/SDT directly from the parse (Section 5.1)	Each reduction emits an instruction using *newTemp* and *gen*; no DAG is built.	Computed again each time they occur (e.g., $b - c$ twice).

SDD/SDT is the *mechanism* for generating code in one pass while parsing; the DAG is what gives *sharing*. Methods 1 and 2 give the same code.

Caution: assignments inside the DAG

If the DAG covers several statements and a variable is *reassigned* in between (e.g., `a = a + 1`), a later use of a is a different value and must not share the old leaf or any node built from it. Compilers handle this by creating a new leaf (a new value number) after each assignment. For single expressions, as in these notes, the issue does not arise.

2.3.2 Addresses and instructions

An **address** can be: (i) a *name* (source variable; a pointer to its symbol-table entry), (ii) a *constant*, or (iii) a compiler-generated *temporary*.

Instruction form	Meaning
`x = y op z`	binary assignment
`x = op y`	unary assignment (`minus`, `not`, conversions)
`x = y`	copy
`goto L`	unconditional jump
`if x goto L` / `ifFalse x goto L`	conditional jump on truth value
`if x relop y goto L`	conditional jump on comparison
`param x`, `call p, n`, `y = call p, n`, `return y`	procedure calls
`x = y[i]`, `x[i] = y`	indexed copy (memory at address $y + i$)
`x = &y`, `x = *y`, `*x = y`	address and pointer assignments
`L:`	label (a symbolic name for an instruction)

Table 3: Common three-address instruction forms.

2.3.3 Representations of three-address code

Three-address code is an *abstract* language: it says *what* the instructions are. A compiler also has to decide *how to store* them in memory, as records in an array. There are two basic record layouts: **quadruples** and **triples**. Each of them represents the complete code on its own. (A third name,

indirect triples, is introduced at the end of this subsection; it is not a different representation of the code but an *add-on to triples* that makes reordering cheap.)

Take the source statement $a = b * -c + b * -c$.

(a) Three-address code

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a  = t5
```

(b) Quadruples – (op, arg1, arg2, result)

	op	arg_1	arg_2	result
0	`minus`	c		t1
1	`*`	b	t1	t2
2	`minus`	c		t3
3	`*`	b	t3	t4
4	`+`	t2	t4	t5
5	`=`	t5		a

(c) Triples – (op, arg1, arg2). There is no result field: another instruction refers to this result by the *position* of the triple, written (i).

	op	arg_1	arg_2
0	`minus`	c	
1	`*`	b	(0)
2	`minus`	c	
3	`*`	b	(2)
4	`+`	(1)	(3)
5	`=`	a	(4)

How to read it. Triple 1 means "multiply b by *the result of triple 0*". Triple 4 adds the results of triples 1 and 3. The temporaries $t_1, \ldots, t_5$ have disappeared: the *position* plays their role.

	Advantage	**Disadvantage**
Quadruples	Every result has an explicit name (a temporary), so an instruction can be moved anywhere without editing any other instruction.	Needs an extra field, and temporaries must be entered in the symbol table.
Triples	No temporaries and no result field: more compact.	A result is identified by its position, so *moving* a triple changes its number and every instruction that refers to it must be edited.

Table 4: Quadruples versus triples.

Indirect triples: triples plus an ordering list. The disadvantage of triples shows up in optimisation, where the compiler often wants to *reorder* instructions. Indirect triples fix it by separating two things that plain triples tie together: *the triples themselves* (kept in a fixed array, numbered once and never renumbered) and *the order of execution* (a separate list of pointers to triples). So indirect triples are not a new way of writing the code; they are triples with one extra list.

Example: evaluate $c * d$ before $a + b$ in $x = (a + b) - (c * d)$

The code is $t_1 = a + b$, $t_2 = c * d$, $t_3 = t_1 - t_2$, $x = t_3$. Reordering the two independent sub-expressions is legal. Compare the three situations (edited fields in red):

Triples (original)

	op	a_1	a_2
0	+	a	b
1	*	c	d
2	-	(0)	(1)
3	=	x	(2)

Triples after reordering

	op	a_1	a_2
0	*	c	d
1	+	a	b
2	-	(1)	(0)
3	=	x	(2)

Indirect triples (reordered)

order list

slot	points to
35	(1)
36	(0)
37	(2)
38	(3)

triples (unchanged)

0	+	a	b
1	*	c	d
2	-	(0)	(1)
3	=	x	(2)

With plain triples the two triples swap places, so *every* reference to them (here in triple 2) must be rewritten. With indirect triples the triples keep their numbers; only the order list changes, and no triple is edited. In a large program the number of references to repair can be huge, which is why optimising compilers prefer the extra list.

Which one should I use?

Quadruples are the simplest to reason about and to optimise, and are the usual textbook choice. Triples save space but are fragile under reordering. Indirect triples keep the compactness of triples and recover the flexibility of quadruples at the price of one extra list. All three encode exactly the same three-address code.

2.3.4 Static single-assignment (SSA) form and the φ-function

The idea. In **SSA form** every variable name is assigned in exactly one place in the program text. If the source assigns to the same variable several times, the compiler gives each assignment its own subscripted name. For straight-line code this is nothing more than renaming:

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

```
p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1
```

On the right p_1, p_2, p_3 are three different values that happened to share the name p on the left. Every *use* now names exactly one *definition*: the p_3 in the last line can only come from `p3 = e - p2`.

Why bother? One definition per name makes optimisation easy

```
a  = 1                a1 = 1
b  = a + 2            b1 = a1 + 2
a  = 10               a2 = 10
c  = a * b            c1 = a2 * b1
```

On the left, to fold `c = a * b` an optimiser must first find out *which* of the two assignments to a reaches that line (a data-flow analysis). On the right the question does not exist: a_2 has a single definition, `a2 = 10`; $b_1 = a_1 + 2 = 3$; hence $c_1 = 10 * 3 = 30$. Once a_1 has no uses left, a quick count shows that `a1 = 1` is dead code and can be deleted. SSA turns "which definition reaches here?" into a lookup by name.

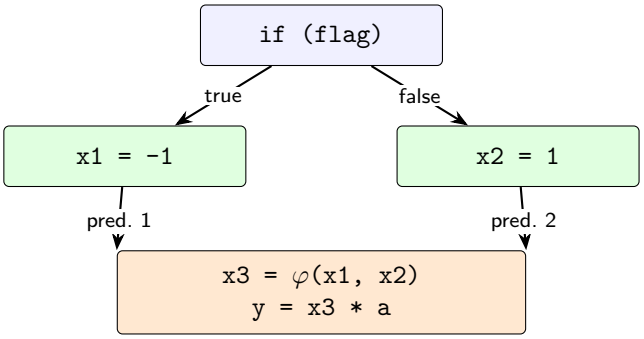
The problem: two paths meet. Straight-line code is easy. The difficulty starts where control flow *joins*:

```
if (flag) x = -1; else x = 1;
y = x * a;
```

Renaming the two assignments gives x_1 (in the **then**-branch) and x_2 (in the **else**-branch). But which name should `y = x * a` use? It depends on `flag`, which is known only at run time, so the compiler cannot choose at compile time. What we need is a *new name* that means "x_1 if we came through the **then**-branch, otherwise x_2". That is exactly what the φ-function provides:

$$x_3 = \varphi(x_1, x_2)$$

```
if (flag) x1 = -1; else x2 = 1;
x3 = phi(x1, x2);
y  = x3 * a;
```



φ takes its *first* operand if control came from pred. 1, its *second* if it came from pred. 2

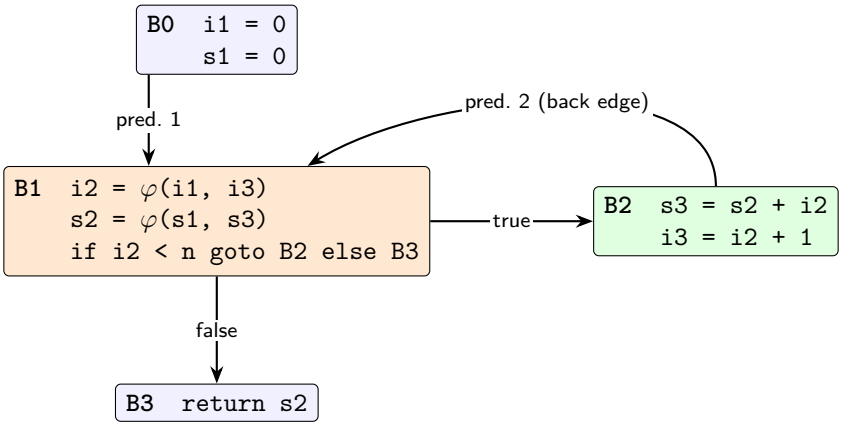
What a φ-function is (and is not)

- It sits at the *start* of a block that has two or more predecessors (a join point).
- $v = \varphi(v_1, \ldots, v_k)$ has one operand per predecessor, *in the same order as the predecessors*. The value of v is the operand that belongs to the edge along which control actually arrived.
- If a block starts with several φ's, they are evaluated *simultaneously* on entry.
- It is **not** an instruction that the machine executes and it does not test anything. It is notation that the compiler uses to keep "one definition per name" true at a join. It is removed again before machine code is generated (see below).

A loop: φ at the loop header. A loop is also a join: the header can be reached from before the loop (the first time) or from the end of the body (every later time). Take

```
i = 0;  s = 0;
while (i < n) { s = s + i;  i = i + 1; }
return s;
```

The variables i and s both have different definitions arriving at the header, so both need a φ there:



Both φ's list the operand from B0 first ("pred. 1") and the operand from B2 second ("pred. 2"). Note that i_3 and s_3 are used by the φ's *before* they are defined in program order; that is fine, because that operand is only chosen when control arrives from B2, i.e. after B2 has run. Trace for $n = 2$:

visit of B1	**arrived from**	i_2	s_2	$i_2 < 2$?	**then B2 computes**
1	B0 (pred. 1)	$i_1 = 0$	$s_1 = 0$	yes	$s_3 = 0 + 0 = 0$, $i_3 = 1$
2	B2 (pred. 2)	$i_3 = 1$	$s_3 = 0$	yes	$s_3 = 0 + 1 = 1$, $i_3 = 2$
3	B2 (pred. 2)	$i_3 = 2$	$s_3 = 1$	no	leave loop: `return s2` $= 1$

The result $1 = 0 + 1$ is the sum of i over $i = 0, 1$, as the source program computes.

Where is a φ needed? Only at a join point, and only for a variable for which *different definitions* arrive along different paths. A variable that has a single reaching definition needs no φ:

```
y = 7;                          y1 = 7;
if (c) x = 1; else x = 2;       if (c) x1 = 1; else x2 = 2;
z = x + y;                      x3 = phi(x1, x2);   z1 = x3 + y1;
```

Here x gets a φ (two definitions meet) but y does not (only y_1 reaches the join, along both paths). Compilers find the exact places with an algorithm based on *dominance frontiers*, which is beyond these notes; the rule of thumb above is enough for exercises.

Leaving SSA. Real processors have no φ instruction. Before code generation each φ is replaced by *copies at the end of the predecessors*: for $x_3 = \varphi(x_1, x_2)$ the first predecessor gets `x3 = x1` and the second gets `x3 = x2`. Often the register allocator gives x_1, x_2, x_3 the same register and the copies disappear.

Common confusions about SSA

- "Single" counts assignments in the *program text*, not executions. In the loop above `i3 = i2 + 1` runs once per iteration, yet it is the only assignment to i_3 in the text.
- φ chooses by *which edge control arrived on*, not by testing a condition or a value.
- The order of the φ operands must match the order of the predecessors. Swapping them silently gives a different program.
- No φ is needed for a variable that is not reassigned before the join.

3 Syntax-Directed Definitions (SDD)

3.1 What is an SDD, and what is it for?

Division of labour. Up to now the course has used a **context-free grammar (CFG)**. A CFG together with a parser answers exactly two questions:

1. *Is this string a legal program?* (**syntax**)
2. *What is its structure?* The answer is the **parse tree**.

That is all the CFG is needed for. Producing the parse tree is the job of **syntax analysis**, and the grammar alone is enough for it.

But a compiler must also know what the program *means*, and a CFG cannot express this. For example, the parse tree of `3 * 5 + 4` shows that $3 * 5$ is grouped first, but nowhere does it contain the number 19. It also contains no machine or three-address instructions. Typical questions that the parse tree cannot answer are:

- What is the **value** of this expression? (calculators, constant folding)
- What is the **type** of this expression, and do the types fit together? (type checking)
- Was this variable **declared** before it is used? (static checks, symbol table)
- In which temporary or variable is its result **stored**, and which **instructions** implement this statement? (code generation)
- To which **label** should control jump if this condition is true? (control flow)

Example: a program that is syntactically perfect but wrong

```
int x;
y = x + 1;
```

With a grammar such as $S \to \textbf{id} = E$; the second line is a legal statement, and the parser happily builds a parse tree for it. Yet the program is wrong, because y was never declared. The grammar cannot say so: to detect the error, the compiler must look y up in a *symbol table*, and that table is information *outside* the tree structure. (Formally, "declared before use" is not a context-free property.) The same applies to `int a; a = "hi";`: the grammar sees only **id** = **string**, not that the types disagree.

The solution: attach information to the tree. A **syntax-directed definition (SDD)** is a CFG *extended* with two things:

> **SDD** = **CFG** + **attributes** (values stored on grammar symbols) + **semantic rules** (how to compute those values from the production used).

The CFG part is unchanged and still defines the syntax. The new parts add meaning. Applying the rules to a parse tree gives an **annotated parse tree**: the same tree with every node carrying its

attribute values. "Syntax-directed" means that the computation is *driven by the syntactic structure*: which rule fires at a node depends on which production was used there.

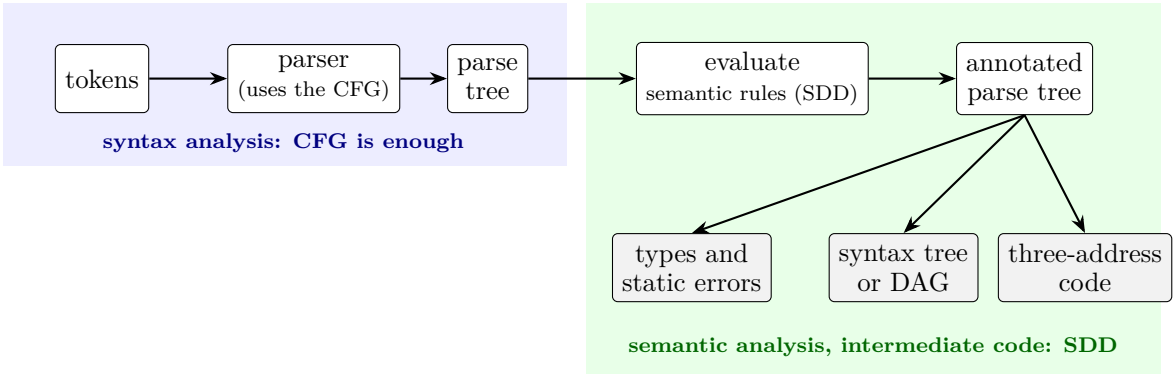


Figure 6: Where the CFG ends and the SDD begins. The parser needs only the grammar; everything that computes *meaning* from the tree is specified by semantic rules.

Do real compilers build the parse tree?

Usually not. The parse tree is a *conceptual* object: it is the structure that the grammar assigns to the input, and it is what an SDD is defined on (annotated parse tree, dependency graph). A real parser seldom stores it:

- A recursive-descent parser follows the derivation through its procedure calls, and an LR parser follows it through its stack and reductions. The tree is traversed implicitly, not built.
- At each reduction (or procedure return) the parser executes the semantic actions, which is exactly the SDT idea. Typically these actions build a **syntax tree (AST)** node directly, e.g. $E.node = \textbf{new}\ Node('+', E_1.node, T.node)$ (Section 3.9).

A parse tree has a node for every grammar symbol, including chain steps such as $E \to T \to F \to \textbf{digit}$ and punctuation such as parentheses. A syntax tree keeps only the operators and operands that matter, so it is much smaller and is the usual input of the later phases (type checking, intermediate-code generation).

So the two statements fit together. *Logically*, the CFG gives the structure and the SDD gives the meaning. *In practice*, the parser's actions (an SDT) build the syntax tree directly and the full parse tree is never created. Later phases then evaluate attributes (for example types) over the syntax tree, which is attribute evaluation again, only on a different tree.

What is it used for? To answer the common question directly: yes. A CFG with its parse tree is enough for *syntax*; an SDD is the tool used to specify *semantic analysis* and *intermediate-code generation*, and related jobs:

Task	Attribute computed	Compiler phase
Evaluate an expression	*val*	calculators, constant folding
Check types, declarations	*type*, symbol-table entry	semantic analysis
Build a syntax tree or DAG	*node* (pointer)	front end, intermediate representation
Generate three-address code	*addr*, *code*	intermediate-code generation
Generate jumps for conditions	*true*, *false*, *next* (labels)	intermediate-code generation

Two remarks. First, the boundary between "semantic analysis" and "code generation" is blurred in practice, because one set of semantic rules often does both at once, for instance a rule that checks the types and emits a conversion in the same step. Second, an SDD is a general method, also used outside compilers (for example to evaluate expressions or to convert one data format to another).

	CFG	**SDD**	**SDT**
What it is	productions only	CFG + attributes + semantic rules	CFG + program fragments (*actions*) in the bodies
Question answered	"Is it legal? What is the structure?"	"What does it mean? What should be computed?"	"*At what moment of the parse* is each computation done?"
Output	parse tree	annotated parse tree	the translation itself (values, tree, code)
Order of work	–	only constrained by the dependencies (a partial order; many orders are legal)	one fixed order: left to right, each action at its position in the body
Used in	syntax analysis	semantic analysis, intermediate-code generation (the *specification*)	the *implementation* inside the parser

Table 5: CFG, SDD and SDT. An SDD says *what* to compute and which orders are legal; an SDT (Section 4) fixes *one* order and the moment in the parse at which each computation runs.

Do we need both SDD and SDT?

No: they are *two levels of description of the same translation*, not two separate tools used side by side.

- The **SDD** is the *specification*. It states what every attribute equals, e.g. $E.val = E_1.val + T.val$, and fixes the *dependencies*: an attribute can be computed only after the attributes it uses. This allows many evaluation orders, and the SDD does not choose among them.
- The **SDT** is the *implementation*. The same computation is written as an action inside the production, e.g. $E \to E_1 + T \ \{ \ E.val = E_1.val + T.val \ \}$, and therefore runs at one fixed point during parsing: the SDT *chooses one* of the orders that the SDD allows.

Normally we first think in terms of an SDD (what should be computed?) and then turn it into an SDT (where do the actions go?). The SDD alone defines the meaning but is not yet a procedure; one can also write an SDT directly, but then the correct ordering is our own responsibility.

So does an SDD say nothing about when? It does say something, but only partially. The class of an SDD tells us which evaluation strategy is guaranteed to work: post-order for S-attributed, one left-to-right depth-first traversal for L-attributed, and a topological order of the dependency graph for any other non-circular SDD. These are the *legal* orders. What the SDD cannot express is (i) the order of computations that do not depend on each other, which matters for side effects such as *print*, *gen* and *addType*, and (ii) *when during compilation* the work is done, for instance during parsing, without storing the tree. An SDT fixes both.

3.2 Attributes

An **attribute** is any value that we attach to a grammar symbol. It helps to picture the parse tree as a tree of *records*: every node has the grammar symbol as its name plus a few fields (the attributes).

If X is a symbol and a is an attribute, we write $X.a$ for the value of a at a particular node labelled X. Attributes can be of any kind: numbers, strings, types, pointers to symbol-table entries, or even whole code sequences.

A note on the symbol names. The letters E, T, F, S, B used in this chapter have *no special meaning in CFGs in general.* They are just names that the author of a particular grammar chose, by convention: E for *expression*, T for *term*, F for *factor*, S for *statement* and B for *boolean expression.* The expression grammar we use for examples is

$$E \to E + T \mid T, \qquad T \to T * F \mid F, \qquad F \to (\,E\,) \mid \textbf{digit},$$

and a different grammar could use completely different names. Which attributes a symbol carries is likewise decided by the grammar designer, and an attribute exists only on the symbols for which it is declared ($E.val$ exists, but $B.val$ does not unless we define it).

Examples of attributes. Typical attributes, with the symbols they are usually given in the grammars of this chapter, are:

- $E.val$: the numeric value of an expression (E, T, F in the grammar above).
- $E.type$: the type of an expression (`int`, `float`, ...).
- $E.addr$: the name or temporary that holds the result of the expression.
- $E.node$: a pointer to the syntax-tree node built for the expression.
- $S.code$: the three-address code generated for a statement (or expression).
- $B.true$, $B.false$: the labels to jump to when a boolean expression is true or false.
- **id**.*lexeme*: the actual characters of an identifier, supplied by the lexer.
- **digit**.*lexval*: the numeric value of a number token, supplied by the lexer.

Different *occurrences* of the same nonterminal in a production are distinguished by subscripts, e.g., in $E \to E_1 + T$ the E on the left is the *head* and E_1 is the E in the body; both have a *val*.

3.3 Semantic rules: a first tiny example

A **semantic rule** is attached to a production and says how to compute one attribute from other attributes *of the symbols in that same production.* For the production $E \to E_1 + T$ the rule

$$E.val = E_1.val + T.val$$

says: "the value of the whole expression is the value of the left sub-expression plus the value of the term." Consider the grammar $E \to E + T \mid T,\ T \to \textbf{digit}$, together with the rules

Production	Semantic rule
$E \to E_1 + T$	$E.val = E_1.val + T.val$
$E \to T$	$E.val = T.val$
$T \to \textbf{digit}$	$T.val = \textbf{digit}.lexval$

Evaluate it on the input `2 + 3`. The parse tree is $E \Rightarrow E_1 + T$ with $E_1 \Rightarrow T \Rightarrow \textbf{digit}(2)$ and $T \Rightarrow \textbf{digit}(3)$. Working *from the leaves upward*:

1. The lexer supplies **digit**.*lexval* = 2 and 3.
2. $T \to \textbf{digit}$ gives $T.val = 2$ (left) and $T.val = 3$ (right).
3. $E \to T$ gives $E_1.val = 2$.
4. $E \to E_1 + T$ gives $E.val = 2 + 3 = 5$.

That is all an SDD does: it decorates the parse tree with attribute values by repeatedly applying the rules.

3.4 Formal definition

Syntax-Directed Definition (SDD)

A **syntax-directed definition** is a context-free grammar together with **attributes** attached to grammar symbols and **semantic rules** attached to productions. For a production $A \to \alpha$ every rule has the form

$$b = f(c_1, c_2, \ldots, c_k)$$

where f is a function and either

(1) b is a **synthesized** attribute of A and $c_1, \ldots, c_k$ are attributes of the symbols in α (the children), or other attributes of A itself (allowed only if this creates no cycle); or

(2) b is an **inherited** attribute of a symbol B in α and $c_1, \ldots, c_k$ are attributes of A (the parent) or of symbols in α (the siblings), or of B itself.

3.5 Synthesized and inherited attributes

Analogy

Think of the parse tree as a company. A **synthesized** attribute is a *report sent up* to the manager: the manager's value is built from the reports of the subordinates (e.g., the value of $2 + 3$ is built from the values of 2 and 3). An **inherited** attribute is an *instruction or information passed down or across*, from a parent or from siblings (e.g., the declared type `float` is passed down to every identifier in `float a, b, c`).

- A **synthesized attribute** at a parse-tree node N is computed from attributes of N's *children*. (Occasionally it also uses another attribute of N itself, e.g. $A.t = A.s + 1$; this is allowed as long as there is no cycle.) Information flows *bottom-up*. Terminals have synthesized attributes only (supplied by the lexer, e.g., **digit**.*lexval*).
- An **inherited attribute** at N is computed from attributes of N's *parent and siblings*. Information flows *top-down / sideways*. Useful when the structure of the parse tree does not match the abstract syntax, e.g., passing a type to a list of identifiers, or passing the label to jump to.

Example: synthesized attributes only (desk calculator)

Production	Semantic rules
$L \to E$ **n**	$L.val = E.val$
$E \to E_1 + T$	$E.val = E_1.val + T.val$
$E \to T$	$E.val = T.val$
$T \to T_1 * F$	$T.val = T_1.val \times F.val$
$T \to F$	$T.val = F.val$
$F \to (\, E \,)$	$F.val = E.val$
$F \to$ **digit**	$F.val =$ **digit**.*lexval*

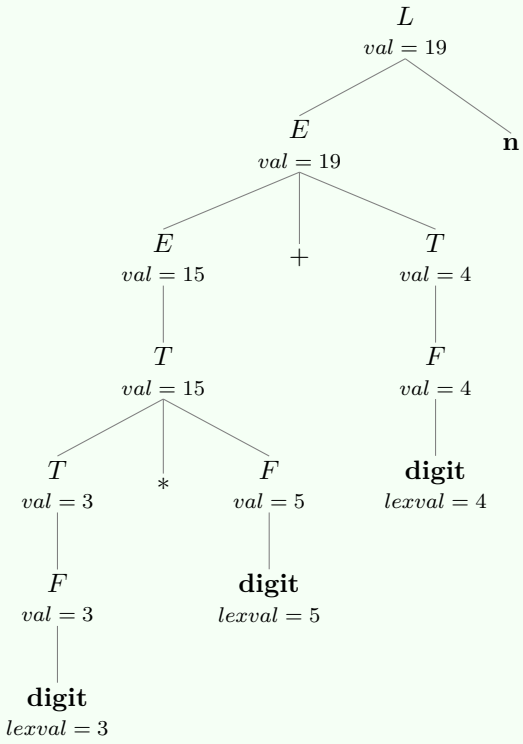


Figure 7: Annotated parse tree for `3*5+4 n`. Each node shows the value of its synthesized attribute *val*.

How to read the figure (evaluation recipe).

1. The lexer supplies the *lexval* of every **digit** leaf (3, 5, 4).
2. Each $F \to$ **digit** copies it into $F.val$; each $T \to F$ copies it up again.
3. $T \to T_1 * F$ is applied once both its children are known: $3 \times 5 = 15$.
4. $E \to E_1 + T$ then adds $15 + 4 = 19$, and $L \to E\,\mathbf{n}$ passes 19 to the root.

Every node is computed *after* its children, i.e., in **post-order**. This is natural for synthesized attributes, and it is exactly the order in which an LR parser performs reductions.

Which evaluation order is needed?

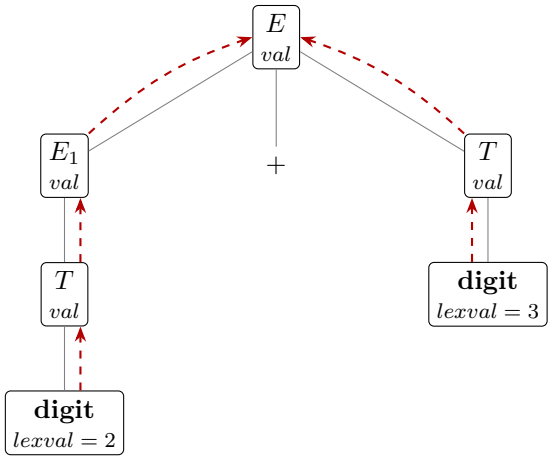
Kind of SDD	Evaluation order	Dependency graph needed?
S-attributed (synthesized only)	post-order (children before parent), i.e. LR reductions	No: every edge points from child to parent, so post-order is already a topological order.
L-attributed	one depth-first, left-to-right traversal	No: inherited attributes depend only on the parent and left siblings.
General (arbitrary mix)	any topological order of the dependency graph	Yes: build the graph, topologically sort it, and reject it if it has a cycle.

So the dependency graph with topological sort is the *general* method; the two special classes are exactly those for which a fixed traversal makes it unnecessary.

3.6 Annotated parse trees and dependency graphs

An **annotated parse tree** shows the attribute values at every node. To evaluate attributes we build a **dependency graph**: one node per attribute instance, and an edge $x \to y$ whenever y is

computed from x. A valid evaluation order is any *topological order* of this graph; if the graph has a cycle, no order exists.



Dashed arrow $x \to y$: y is computed from x. One valid order: both *lexval*s, then the two $T.val$, then $E_1.val$, then $E.val$.

Figure 8: Dependency graph for `2 + 3` under the SDD of this section (every attribute is synthesized, so every arrow points *up*).

A circular definition

For $A \to B$ with the rules $A.s = B.i + 1$ and $B.i = A.s \times 2$, the value of $A.s$ needs $B.i$, which needs $A.s$. The dependency graph has a *cycle*, so no evaluation order exists. A good SDD must be *non-circular*; the classes S-attributed and L-attributed below guarantee this automatically.

Side effects

A semantic rule may also do something besides computing a value, e.g., $addType(entry, t)$ writes into the symbol table, *print* writes to the output, and *gen* emits an instruction. Such rules are called **side effects**; they are fine as long as the order in which they run does not change the result (or the order is controlled, which is the job of an SDT).

Example: inherited attributes (declaration list)

Production	Semantic rules
$D \to T\ L$	$L.inh = T.type$
$T \to$ **int**	$T.type = integer$
$T \to$ **float**	$T.type = float$
$L \to L_1$, **id**	$L_1.inh = L.inh$; $addType(\textbf{id}.entry,\ L.inh)$
$L \to$ **id**	$addType(\textbf{id}.entry,\ L.inh)$

For the input `float` $\texttt{id}_1$`,` $\texttt{id}_2$ the dashed arrows in Fig. 9 are the dependency-graph edges. The type is passed *down and across* from T to L, then down through the list to each identifier.

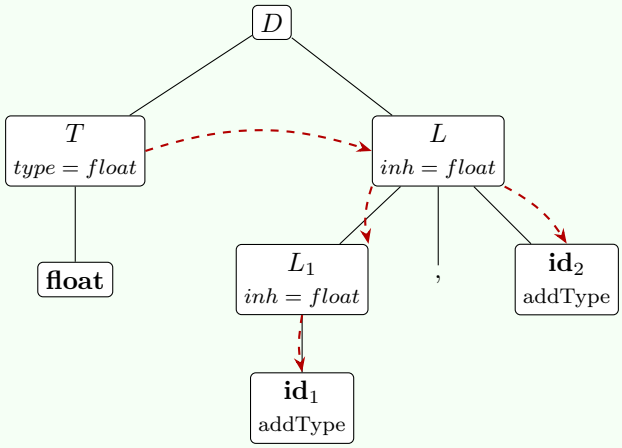


Figure 9: Parse tree for `float id1, id2` with dependency edges (dashed) for the inherited attribute *inh*.

One valid evaluation order (any topological order of the dependency graph works):

(1) $T.type = float$ ($T \to$ **float**; depends on nothing)
(2) $L.inh = T.type$ ($D \to T\ L$)
(3) $L_1.inh = L.inh$ ($L \to L_1$, $\mathbf{id}_2$)
(4) $addType(\mathbf{id}_2.entry,\ L.inh)$ (the identifier after the comma)
(5) $addType(\mathbf{id}_1.entry,\ L_1.inh)$ ($L_1 \to \mathbf{id}_1$)

Steps (4) and (5) do not depend on each other, so they may be swapped; both identifiers end up with type *float* either way.

3.7 Classes of SDDs that are easy to implement

S-attributed and L-attributed definitions

- An SDD is **S-attributed** if *every attribute is synthesized*. It can be evaluated in any bottom-up order, e.g., during LR parsing, at the time of reduction.
- An SDD is **L-attributed** if each attribute is either synthesized, or inherited such that, for a production $A \to X_1 X_2 \cdots X_n$, the inherited attribute of X_i depends only on:
  (a) inherited attributes of A, and
  (b) attributes (inherited or synthesized) of $X_1, \ldots, X_{i-1}$ (the symbols to its *left*), and
  (c) attributes of X_i itself, provided there is no cycle.

  Evaluation is possible in a single left-to-right depth-first traversal (e.g., during top-down/LL parsing).

Every S-attributed SDD is also L-attributed.

Not L-attributed

For $A \to B\,C$, the rule $C.i = B.s$ is fine (the inherited attribute of C uses its *left* sibling B). But the rule $B.i = f(C.x)$ makes the inherited attribute of B depend on its *right* sibling C, so the SDD is *not* L-attributed.

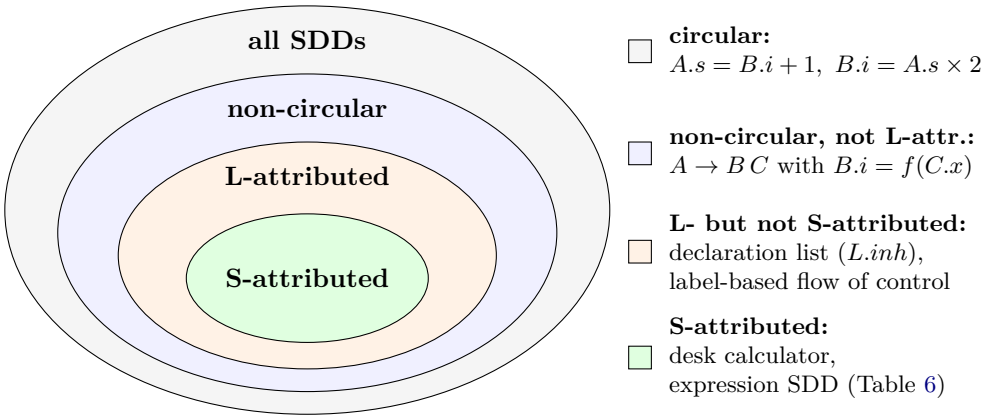


Figure 10: How the classes of SDDs nest, with an example of each.

3.8 More examples: L-attributed and general (non-circular) SDDs

Most examples so far are S-attributed. Here are two examples with inherited attributes: one that is L-attributed, and one that is neither S- nor L-attributed, so that a dependency graph and a topological order are really needed.

How to classify an SDD

1. Is every attribute synthesized? ⇒ **S-attributed**: evaluate in post-order (LR reductions).
2. Otherwise, look at *every inherited attribute* $X_i.a$ in a production $A \to X_1 \cdots X_n$. Does its rule use only inherited attributes of A, attributes of $X_1, \ldots, X_{i-1}$ (left siblings), or other attributes of X_i itself? If this holds for all of them ⇒ **L-attributed**: one depth-first left-to-right traversal suffices.
3. If some inherited attribute uses a *right* sibling (or a synthesized attribute that would have to come from the right), the SDD is **not L-attributed**. Draw the dependency graph. If it has no cycle, the SDD is **non-circular** and any topological order evaluates it; if it has a cycle, the SDD is erroneous.

Example 1 (L-attributed, not S-attributed): value of a binary fraction

We evaluate a binary fraction such as `.101` $= \frac{1}{2} + \frac{1}{8} = 0.625$. The weight of a bit depends on its *position counted from the point*, which is passed *down and to the right* as an inherited attribute *pos*.

Production	Semantic rules
$S \to {}'.'\ L$	$L.pos = 1; \quad S.val = L.val$
$L \to B\ L_1$	$B.pos = L.pos; \quad L_1.pos = L.pos + 1; \quad L.val = B.val + L_1.val$
$L \to \varepsilon$	$L.val = 0$
$B \to 0$	$B.val = 0$
$B \to 1$	$B.val = 2^{-B.pos}$

Why L-attributed. The inherited attributes are $L.pos$, $B.pos$ and $L_1.pos$. In $L \to B\,L_1$, $B.pos$ and $L_1.pos$ use only $L.pos$, the inherited attribute of the head, and in S the rule for $L.pos$ uses nothing. No inherited attribute depends on a symbol to its right. It is not S-attributed because *pos* is inherited.

Evaluation for `.101`. A single depth-first left-to-right traversal works. Writing L_a, L_b, L_c, L_d

for the four occurrences of L from left to right (so $L_a \to B_1 L_b$, $L_b \to B_2 L_c$, $L_c \to B_3 L_d$, $L_d \to \varepsilon$):

Step	Rule used	Value
1	$L_a.pos = 1 \quad (S \to {'.'}L)$	1
2	$B_1.pos = L_a.pos,\ \ L_b.pos = L_a.pos + 1$	1, 2
3	$B_1 \to 1$: $B_1.val = 2^{-1}$	0.5
4	$B_2.pos = 2,\ \ L_c.pos = 3;\ \ B_2 \to 0$: $B_2.val = 0$	0
5	$B_3.pos = 3,\ \ L_d.pos = 4;\ \ B_3 \to 1$: $B_3.val = 2^{-3}$	0.125
6	$L_d \to \varepsilon$: $L_d.val = 0$	0
7	going back up: $L_c.val = 0.125 + 0,\ \ L_b.val = 0 + 0.125,\ \ L_a.val = 0.5 + 0.125$	0.625
8	$S.val = L_a.val$	0.625

As an SDT. Following the conversion rules of Section 4.7, each inherited attribute is computed by an action placed immediately before the symbol that uses it; this gives actions at the left end of a body and in the middle:

$$S \to {'.'}\ \{\ L.pos = 1\ \}\ L\ \{\ S.val = L.val\ \}$$
$$L \to \{\ B.pos = L.pos\ \}\ B\ \{\ L_1.pos = L.pos + 1\ \}\ L_1\ \{\ L.val = B.val + L_1.val\ \}$$
$$\mid\ \varepsilon\ \{\ L.val = 0\ \}$$
$$B \to 0\ \{\ B.val = 0\ \}\ \mid\ 1\ \{\ B.val = 2^{-B.pos}\ \}$$

Example 2 (neither S- nor L-attributed, but non-circular): value of a binary integer

Now evaluate an ordinary binary integer such as `101` $= 5$. The weight of a bit is 2^{pos} where *pos* is the number of bits *to its right*. This depends on the *length of the rest of the string*, which is known only after the right part has been examined.

Production	Semantic rules
$L \to B\ L_1$	$B.pos = L_1.len;\quad L.len = L_1.len + 1;\quad L.val = B.val + L_1.val$
$L \to B$	$B.pos = 0;\quad L.len = 1;\quad L.val = B.val$
$B \to 0$	$B.val = 0$
$B \to 1$	$B.val = 2^{B.pos}$

Classification. *pos* is inherited, so the SDD is not S-attributed. In $L \to B\,L_1$ the inherited attribute $B.pos$ is computed from $L_1.len$, an attribute of the *right* sibling, so it is not L-attributed either: a left-to-right traversal reaches B before it knows $L_1.len$. It is nevertheless *non-circular*: *len* depends only on *len* of the part below it, *pos* depends only on *len*, and *val* depends on *pos* and on *val* further down. So we use a dependency graph.

Dependency graph for `101`. The parse tree is $L_a \to B_1 L_b$, $L_b \to B_2 L_c$, $L_c \to B_3$, with $B_1 \to 1$, $B_2 \to 0$, $B_3 \to 1$. Each box in Figure 11 is one attribute instance with its final value; an arrow $x \to y$ means that y is computed from x. The small red numbers give one valid topological order.

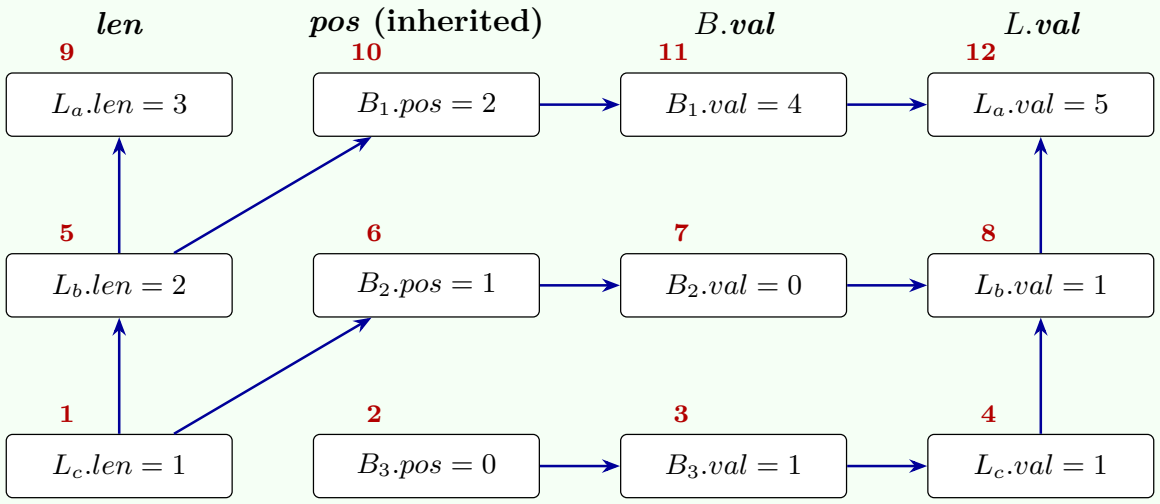


Figure 11: Dependency graph for `101` (value 5). *len* flows upward, then crosses over into the inherited *pos* of the left sibling B, then *val* flows upward. The red numbers are one valid topological order.

Topological order (solution).

(1) $L_c.len = 1$ and $B_3.pos = 0$ (rule $L \to B$; no dependence on anything).
(2) $B_3.val = 2^0 = 1$ $(B_3 \to 1)$, then $L_c.val = 1$.
(3) $L_b.len = L_c.len + 1 = 2$ and $B_2.pos = L_c.len = 1$.
(4) $B_2.val = 0$ $(B_2 \to 0)$, then $L_b.val = 0 + 1 = 1$.
(5) $L_a.len = L_b.len + 1 = 3$ and $B_1.pos = L_b.len = 2$.
(6) $B_1.val = 2^2 = 4$ $(B_1 \to 1)$, then $L_a.val = 4 + 1 = 5$.

Other orders are also valid, e.g. all three *len* values first. In practice a compiler evaluates such an SDD in *several passes over the parse tree*: a bottom-up pass for *len*, a top-down pass for *pos*, and a bottom-up pass for *val*.

Remark. The difficulty comes from the right-recursive grammar. With the left-recursive grammar $L \to L_1 B \mid B$ and $L.val = 2 \times L_1.val + B.val$ the same value is computed by an S-attributed SDD, with no *pos* and no *len*. Choosing the grammar well can remove the need for a dependency graph.

3.9 SDD for constructing syntax trees

Syntax trees can be built using synthesized attribute *node*, which points to the root of a (sub)tree, and constructor functions $Node(op, c_1, \dots)$ and $Leaf(\textbf{id}, entry)$ / $Leaf(\textbf{num}, val)$.

Production	Semantic rules
$E \to E_1 + T$	$E.node =$ **new** $Node('+', E_1.node, T.node)$
$E \to E_1 - T$	$E.node =$ **new** $Node('-', E_1.node, T.node)$
$E \to T$	$E.node = T.node$
$T \to (\,E\,)$	$T.node = E.node$
$T \to$ **id**	$T.node =$ **new** $Leaf(\textbf{id}, \textbf{id}.entry)$
$T \to$ **num**	$T.node =$ **new** $Leaf(\textbf{num}, \textbf{num}.val)$

This is S-attributed. If the grammar is made LL by eliminating left recursion, the same tree is built with an inherited attribute (see the L-attributed SDT in Section 4.7).

3.10 SDD for constructing DAGs directly

The problem with the two-step method. In Section 2.2.1 we first built the complete syntax tree and then walked over it in post-order to obtain the DAG. That works, but it builds a structure (the tree) that is thrown away immediately. An SDD lets us skip the tree: the parser creates DAG nodes *while it parses*, and the tree never exists in memory.

The key observation. The SDD of the previous subsection already has the right shape. Its rules for $E \to E_1 + T$ and $T \to T_1 * F$ create a node *after* the nodes of the children are known, and a bottom-up parser performs reductions in exactly that (post-)order, which is also the order used by the value-number method. All we change is *what the constructor functions do*:

- In the tree SDD, **new** $Node(op, l, r)$ *always* allocates a fresh node.
- In the DAG SDD, $Node(op, l, r)$ first *searches* for an existing record with the same (op, l, r) and allocates a new one only if there is none. *Leaf* behaves in the same way.

Both functions return a **value number**, so the attribute $E.node$ is now simply *an integer: the index of the DAG node* (instead of a pointer to a tree node). The attribute rules themselves are unchanged.

The SDD. To have a multiplication operator we use the usual three-level expression grammar (E expression, T term, F factor), and a single synthesized attribute *node*:

Production	Semantic rule
$E \to E_1 + T$	$E.node = Node('+',\ E_1.node,\ T.node)$
$E \to E_1 - T$	$E.node = Node('-',\ E_1.node,\ T.node)$
$E \to T$	$E.node = T.node$
$T \to T_1 * F$	$T.node = Node('*',\ T_1.node,\ F.node)$
$T \to F$	$T.node = F.node$
$F \to (\ E\ )$	$F.node = E.node$
$F \to$ **id**	$F.node = Leaf(\textbf{id},\ \textbf{id}.entry)$
$F \to$ **num**	$F.node = Leaf(\textbf{num},\ \textbf{num}.val)$

The rules that only *copy* a value ($E \to T$, $T \to F$, $F \to (E)$) create nothing: parentheses affect only the *shape* of the DAG, they do not produce a node of their own. Only the rules containing *Node* or *Leaf* touch the array.

The two constructor functions. The array of records and a hash table keyed on the *signature* of a record are *global data*, shared by all semantic rules for the whole input:

```
record table[];                 // table[v] = record of the node with value number v
hash   index;                   // signature -> value number

int Leaf(kind, x) {             // kind is id or num; x is the entry or the value
    sig = (kind, x);
    if (index has sig) return index[sig];     // already exists: share it
    v = append(table, sig);                   // new record at the end
    index[sig] = v;
    return v;
}

int Node(op, l, r) {            // l and r are value numbers of the children
    sig = (op, l, r);
```

```
    if (index has sig) return index[sig];     // common sub-expression found
    v = append(table, sig);
    index[sig] = v;
    return v;
}
```

Compare with the `findOrCreate` routines of Section 2.2.1: they are the same functions. The only new idea is *who calls them*: the semantic rules of the grammar, not a tree-walking procedure.

How does the SDD say "tree" or "DAG"?

The *rules* cannot tell the difference: the DAG SDD above and the syntax-tree SDD of the previous subsection have the same productions, the same attribute *node*, and the same dependency graph. Both are S-attributed. What differs is the **meaning of the constructor functions** that the rules call, and that is part of the definition of the SDD, in the same way that the meaning of *gen* or *addType* is:

- **new** $Node(op, l, r)$ *always allocates* a fresh node. Equal sub-expressions get different nodes, so the result is a **syntax tree**.
- $Node(op, l, r)$ *looks for a record with the same* (op, l, r) *first* and allocates only if none exists. Equal sub-expressions get the same node, so the result is a **DAG**.

So whoever writes the SDD must state which of the two is meant. In these notes we use the keyword **new** for "always create" and plain *Node*, *Leaf* for "find or create". Nothing else changes: every later phase simply receives the attribute *node* and does not care whether some nodes are shared (except that shared nodes need to be translated only once).

Why this is a legitimate SDD

Node and *Leaf* have a *side effect* (they change the global table), which Section 3.1 allowed under one condition: the order of the calls must not change the result. Here it is fixed by the dependencies: a parent's *Node* call needs the value numbers of its children, so those calls are made first. The SDD is S-attributed, so it can be evaluated during LR parsing, with each rule executed at the reduction by which its production is recognised.

Example: input $(a + b) * (a + b)$

The table lists the reductions in the order the parser performs them, but only those that call *Leaf* or *Node*. The omitted reductions ($T \to F$, $E \to T$, $F \to (E)$) just copy the value number upward.

Step	Reduction	Call	Result	Value no.
1	$F \to$ **id** (first a)	$Leaf(\mathbf{id}, a)$	new record	1
2	$F \to$ **id** (first b)	$Leaf(\mathbf{id}, b)$	new record	2
3	$E \to E_1 + T$	$Node(+, 1, 2)$	new record	3
4	$F \to$ **id** (second a)	$Leaf(\mathbf{id}, a)$	**found**	1
5	$F \to$ **id** (second b)	$Leaf(\mathbf{id}, b)$	**found**	2
6	$E \to E_1 + T$	$Node(+, 1, 2)$	**found: shared**	3
7	$T \to T_1 * F$	$Node(*, 3, 3)$	new record	4

The attribute of the root E is $E.node = 4$: the root of a DAG with four records, $1 : a$, $2 : b$, $3 : (+, 1, 2)$, $4 : (*, 3, 3)$. At step 6 the second copy of $a + b$ is recognised as the first and no new node is made; this is the sharing we wanted. The tree version would have created seven nodes.

For the larger expression $a + a * (b - c) + (b - c) * d$ the sequence of *Leaf* and *Node* calls is exactly the sequence of requests in Table 1. This is no accident: the value-number method visits the tree in post-order, and an LR parser reduces in post-order too. Both methods therefore build the *same DAG with the same value numbers*, and the final $E.node = 9$ identifies its root.

Tree-first versus direct construction.

	Tree first, then DAG (Section 2.2.1)	**Directly with an SDD**
Needs a syntax tree in memory	yes	no
Who calls find-or-create	a tree-walking procedure	the semantic rules, at reductions
Order of calls	post-order of the tree	order of the parser's reductions (the same)
Resulting DAG	identical	identical

Two details.

- **Commutative operators.** If *Node* stores the children of $+$ and $*$ in a canonical order (smaller value number first), then $a + b$ and $b + a$ are shared as well; without this, they are treated as different expressions (Section 2.2.1).
- **Generating code at the same time.** Whenever *Node creates* a record, it can also emit one three-address instruction for it, with a fresh temporary; when it *finds* an existing record, it emits nothing. This is method 2 in the comparison of Section 2.3.1 (the paragraph "Three ways to obtain three-address code"), and it produces code in which $b - c$ is computed only once.

4 Syntax-Directed Translation Schemes (SDT)

4.1 From specification to implementation

An SDD tells us *what* every attribute must equal, but it does not say *when* each rule should run. A real compiler, however, runs the rules *while it is parsing*, in one definite order, and often without ever building the whole parse tree. A **syntax-directed translation scheme (SDT)** solves this: it takes the program fragments of the rules and writes them *inside the production body, at the exact position where they should run.*

The same rule written both ways:

SDD (specification)	**SDT** (implementation)
$E \to E_1 + T$ rule: $E.val = E_1.val + T.val$	$E \to E_1 + T \quad \{\ E.val = E_1.val + T.val\ \}$

	SDD	**SDT**
Nature	High-level specification: rules attached to productions	Concrete implementation: actions embedded in productions
Order of evaluation	Only constrained by the dependencies: any topological order is legal (post-order, left-to-right DFS, ... depending on the class)	One explicit order: actions run in the left-to-right order of the production body, at the moment the parser reaches them
Rules/actions	Written next to the production	Written in braces {...} inside the body
Used for	Designing and reasoning about the translation	Building the actual translator

4.2 Definition and the meaning of an action's position

Syntax-Directed Translation Scheme

A **syntax-directed translation scheme** (SDT) is a context-free grammar with **program fragments** (*semantic actions*) embedded within the production bodies, written in braces. The position of an action defines *when* it is executed during parsing.

$$B \to X\ \{a\}\ Y$$

Here action a is executed after X has been recognised (and before Y).

How an SDT is executed. Imagine the parse tree in which every action appears as an *extra child*, exactly at the position where it is written in the production. Perform a **depth-first traversal from left to right**; whenever the traversal reaches an action, it is executed. Figure 12 shows this for a scheme that translates infix to postfix: `expr → expr + term {print('+')} | expr - term {print('-')} | term` and `term → digit {print(digit)}`.

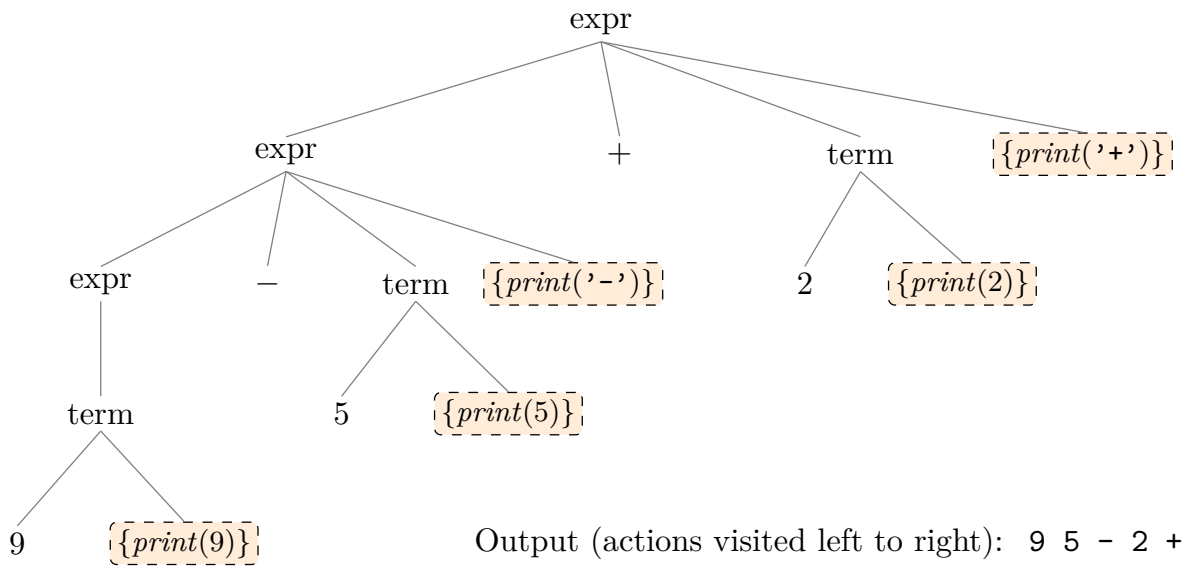


Figure 12: Parse tree of `9-5+2` with actions (dashed boxes) as extra children. Visiting the tree depth-first, left to right, executes the actions in the order 9, 5, −, 2, +, producing the postfix form.

In practice the parser does not need to build this tree. A top-down parser executes an action at the corresponding point of its recursive-descent procedure. A bottom-up (LR) parser can only act

when it *reduces*: an action at the right end of a body runs at that reduction (a postfix SDT, next subsection), while an action in the middle of a body needs a marker nonterminal (Section 4.5).

4.3 Postfix translation schemes

If the grammar is LR-parsable and the SDD is **S-attributed**, we can build an SDT with every action at the *right end* of its production (a **postfix SDT**). The actions then execute at *reduction time*, so the attribute of each grammar symbol can be stored beside it on the parser stack.

Postfix SDT for the desk calculator

$L \to E\ \mathbf{n}$	$\{\ print(E.val)\ \}$
$E \to E_1 + T$	$\{\ E.val = E_1.val + T.val\ \}$
$E \to T$	$\{\ E.val = T.val\ \}$
$T \to T_1 * F$	$\{\ T.val = T_1.val \times F.val\ \}$
$T \to F$	$\{\ T.val = F.val\ \}$
$F \to (\ E\ )$	$\{\ F.val = E.val\ \}$
$F \to \mathbf{digit}$	$\{\ F.val = \mathbf{digit}.lexval\ \}$

Implementation on the LR parse stack. Each stack entry holds a grammar symbol and its attribute value. When $A \to XYZ$ is reduced, the actions read the values in the top three entries (`stack[top-2].val`, `stack[top-1].val`, `stack[top].val`), compute $A.val$, pop three entries, and push A with the new value.

Stack (symbol:val)	Remaining input	Action
\$	`3 * 5 + 4 n`	shift
\$ **digit**:3	`* 5 + 4 n`	reduce $F \to$ **digit**
\$ F:3	`* 5 + 4 n`	reduce $T \to F$
\$ T:3	`* 5 + 4 n`	shift $*$, shift **digit**
\$ T:3 $*$ **digit**:5	`+ 4 n`	reduce $F \to$ **digit**
\$ T:3 $*$ F:5	`+ 4 n`	reduce $T \to T * F$ $\quad(3 \times 5 = 15)$
\$ T:15	`+ 4 n`	reduce $E \to T$
\$ E:15	`+ 4 n`	shift $+$, shift **digit**, reduce to T:4
\$ E:15 $+$ T:4	`n`	reduce $E \to E + T$ $\quad(15 + 4 = 19)$
\$ E:19	`n`	shift **n**, reduce $L \to E\,\mathbf{n}$, print 19

4.4 SDTs with actions inside productions

When an action is in the middle of a production, it is performed *immediately after all grammar symbols to its left have been matched*. A top-down parser does exactly that: the action is just a statement placed at that point of the recursive-descent procedure. An LR parser cannot: it only takes decisions at reductions, so a mid-production action must first be turned into a reduction using a marker nonterminal (Section 4.5).

Where the action is	Top-down (LL / recursive descent)	Bottom-up (LR)
At the right end	run after the last symbol of the body is matched	run at the reduction of the body
In the middle	run at that point of the procedure	not possible directly: introduce a marker $M \to \varepsilon$

Example: infix to postfix (action placed after the operand)

$$E \to T\ R$$
$$R \to +\ T\ \{\ print('+')\ \}\ R_1 \mid -\ T\ \{\ print('-')\ \}\ R_1 \mid \varepsilon$$
$$T \to \textbf{num}\ \{\ print(\textbf{num}.val)\ \}$$

For `9-5+2` the output is `95-2+`.

Worked example: actions in the middle of productions (declarations)

We want to give every identifier in a declaration such as `float x, y` its type. The type is known only after T has been parsed, and the identifiers must receive it *as they are read*. This is why the actions cannot all stand at the end of a production.

$$D \to T\ \textbf{id}\ \{\ L.inh = T.type;\ \ addType(\textbf{id}.entry,\ T.type)\ \}\ L$$
$$T \to \textbf{int}\ \{\ T.type = integer\ \}\ \mid\ \textbf{float}\ \{\ T.type = float\ \}$$
$$L \to ',' \ \textbf{id}\ \{\ addType(\textbf{id}.entry,\ L.inh);\ L_1.inh = L.inh\ \}\ L_1\ \mid\ \varepsilon$$

The actions of T are at the right end of their bodies, but the actions in D and in L are in the *middle*: each is followed by the nonterminal (L or L_1) that needs the inherited attribute it computes.

Solution for `float x, y`. Figure 13 shows the parse tree with each action as an extra child. A depth-first traversal from left to right executes the actions in the order of the bold numbers.

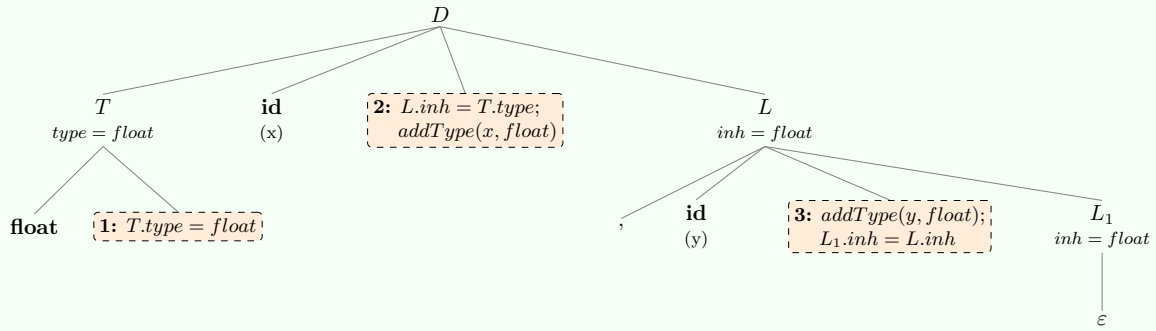


Figure 13: Parse tree of `float x, y` with the actions as extra children (dashed boxes). The bold numbers give the execution order in a left-to-right depth-first traversal.

(1) Reaching the end of $T \to \textbf{float}$: $T.type = float$.
(2) Back in D, after T and **id** (x) have been visited: $L.inh = float$ and $addType(x, float)$.
(3) In L, after `,` and **id** (y): $addType(y, float)$ and $L_1.inh = float$. The remaining $L_1 \to \varepsilon$ has no action.

Result: the symbol table says $x : float$ and $y : float$.

Why the position matters. If the action of D were written at the *end*, after L, then L would be processed before $L.inh$ has a value, and $addType(y, \dots)$ would use an undefined type. A mid-production action is therefore the way an SDT guarantees that an inherited attribute is computed *before* the symbol that uses it. A top-down parser executes these actions as statements of its recursive-descent procedures; an LR parser needs marker nonterminals (Section 4.5).

Example: actions at the left end of a production body

An action may be the very first item of a body. (An action cannot be attached to the head of a production; "left" here means the left end of the body.) Such an action runs *before any symbol of the body is matched*, so it can use only the *inherited* attributes of the head. The scheme below prints the nesting depth of every opening parenthesis of a balanced string; $P.d$ is an inherited attribute holding the depth of the current position.

$$S \to \{\ P.d = 0\ \}\ P$$
$$P \to \{\ print(P.d)\ \}\ '('\ \{\ P_1.d = P.d + 1\ \}\ P_1\ ')'\ \{\ P_2.d = P.d\ \}\ P_2 \quad | \quad \varepsilon$$

Here the first action of S *initialises* the inherited attribute of P, and the first action of P *reads* $P.d$, which was set by an action before P was entered. For the input `(()())` the output is `0 1 1`.

4.5 Marker nonterminals (actions in the middle, for bottom-up parsing)

Suppose an action must appear in the middle of a production, e.g., $B \to X\ \{a\}\ Y$. For an LR parser we replace the action by a **marker nonterminal** M with an empty production:

$$B \to X\ M\ Y, \qquad M \to \varepsilon\ \{a\}$$

The action a is then executed upon reducing $M \to \varepsilon$, which happens exactly when X is on top of the stack and Y has not been read yet.

Example: the infix-to-postfix scheme under LR parsing

The top-down scheme $R \to + \ T\ \{print('+')\}\ R_1$ has its action in the middle. For an LR parser we write

$$R \to + \ T\ M\ R_1, \qquad M \to \varepsilon\ \{\ print('+')\ \}$$

When $M \to \varepsilon$ is reduced, the operand T has just been reduced (and printed), so the output order is unchanged: $9\,5\ -\ 2\,+$ for `9-5+2`.

Markers can create conflicts

A marker forces the parser to decide to reduce $M \to \varepsilon$ *before* it knows which production will finally be used. If two productions start with the same symbols, say $S \to X\,M\,Y \mid X\,Z$, the parser must choose, using only the next input token, whether to reduce M. If Y and Z can start with the same token, the result is a conflict and the grammar is no longer LR(1). The markers in backpatching (M before a statement, N before **else**) are safe because they always occur at a point where the parser can already tell what is coming.

We will use markers M and N extensively in backpatching (Section 8).

4.6 Eliminating left recursion from an SDT

A top-down parser cannot use a left-recursive grammar such as $E \to E_1 + T$, but the *translation* we want (a left-associative tree or value) is naturally defined by that grammar. The fix is to rewrite the grammar *and* the actions together, using an inherited attribute that carries the result built so far.

General transformation

$$A \to A_1\, Y \;\{\; A.a = g(A_1.a,\; Y.y) \;\} \;\mid\; X \;\{\; A.a = f(X.x) \;\}$$

becomes

$$A \to X \;\{\; R.i = f(X.x) \;\}\; R \;\{\; A.a = R.s \;\}$$
$$R \to Y \;\{\; R_1.i = g(R.i,\; Y.y) \;\}\; R_1 \;\{\; R.s = R_1.s \;\} \;\mid\; \varepsilon \;\{\; R.s = R.i \;\}$$

$R.i$ (inherited) carries the value accumulated *so far*; $R.s$ (synthesized) carries the final result back to the top.

Example: evaluating $9 - 5 + 2$ with a left-recursion-free grammar

The left-recursive grammar $E \to E_1 + T \mid E_1 - T \mid T$, $T \to \textbf{num}$, with $E.val = E_1.val \pm T.val$, becomes

$$E \to T \;\{\; R.i = T.val \;\}\; R \;\{\; E.val = R.s \;\}$$
$$R \to + \; T \;\{\; R_1.i = R.i + T.val \;\}\; R_1 \;\{\; R.s = R_1.s \;\}$$
$$\quad \mid \; - \; T \;\{\; R_1.i = R.i - T.val \;\}\; R_1 \;\{\; R.s = R_1.s \;\} \;\mid\; \varepsilon \;\{\; R.s = R.i \;\}$$
$$T \to \textbf{num} \;\{\; T.val = \textbf{num}.lexval \;\}$$

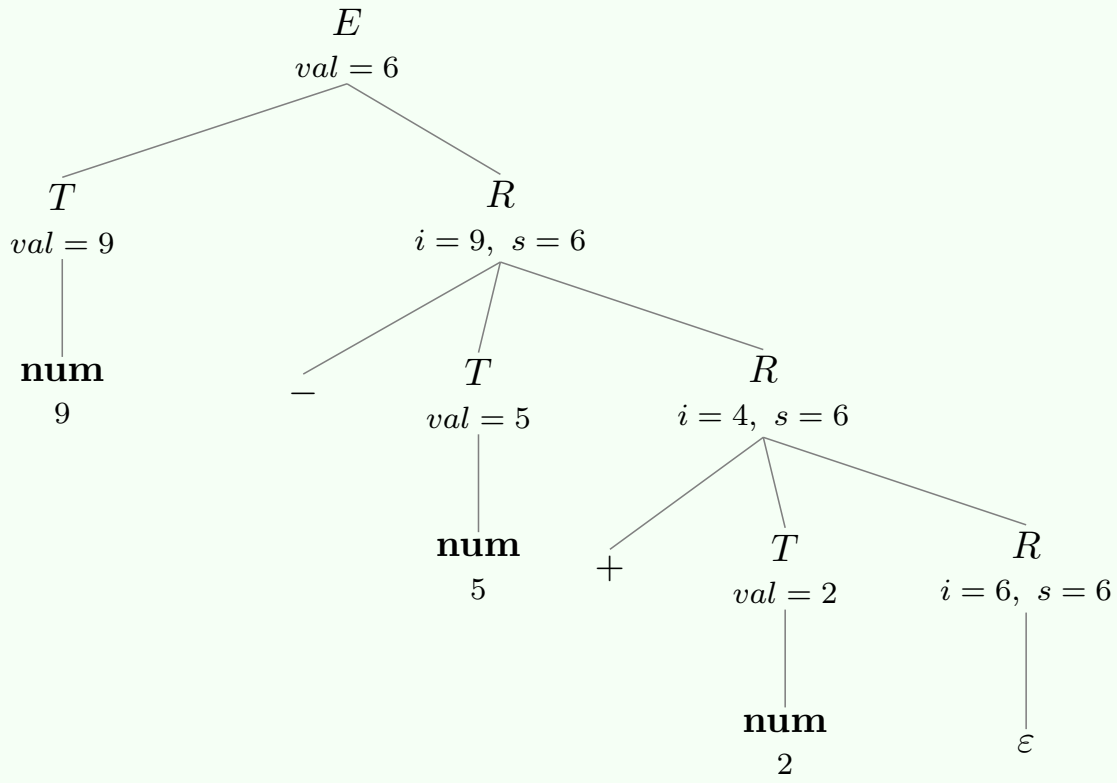


Figure 14: Annotated parse tree for `9-5+2`. The inherited $R.i$ flows down and to the right ($9 \to 4 \to 6$: first $9-5$, then $4+2$); the synthesized $R.s$ then carries the final 6 back up to $E.val$.

The tree leans to the right, but the arithmetic is still left-associative: $(9-5)+2=6$, not $9-(5+2)=2$.

4.7 Converting an L-attributed SDD into an SDT

Rules for converting an L-attributed SDD to an SDT

1. **Inherited attribute** of a nonterminal X in the body: embed the action that computes it *immediately before* X in the body. If several inherited attributes of X depend on one another, order the actions appropriately.
2. **Synthesized attribute** of the head A: place the action that computes it at the *end* of the production body.

Example: syntax tree for a left-recursion-free grammar

After removing left recursion from $E \to E_1 + T \mid E_1 - T \mid T$:

$$E \to T \ \{\ R.i = T.node\ \}\ R\ \{\ E.node = R.s\ \}$$
$$R \to + \ T\ \{\ R_1.i = \textbf{new}\ Node('+', R.i, T.node)\ \}\ R_1\ \{\ R.s = R_1.s\ \}$$
$$R \to - \ T\ \{\ R_1.i = \textbf{new}\ Node('-', R.i, T.node)\ \}\ R_1\ \{\ R.s = R_1.s\ \}$$
$$R \to \varepsilon\ \{\ R.s = R.i\ \}$$
$$T \to \textbf{id}\ \{\ T.node = \textbf{new}\ Leaf(\textbf{id}, \textbf{id}.entry)\ \}$$
$$T \to \textbf{num}\ \{\ T.node = \textbf{new}\ Leaf(\textbf{num}, \textbf{num}.val)\ \}$$

Here $R.i$ (inherited) carries the tree built *so far*; $R.s$ (synthesized) returns the final tree. For `a - 4 + c` the tree $((a-4)+c)$ is produced with the correct left-associativity.

5 Translation of Expressions and Assignments

5.1 SDD for three-address code

We use the following:

- $E.addr$: the address (name, constant, or temporary) that holds the value of E.
- $E.code$ (or $S.code$): the three-address instructions that evaluate E (or execute S).
- $newTemp()$ (written **new** $Temp$): returns a fresh temporary $t_1, t_2, \ldots$ in successive calls.
- **gen**$(\cdots)$: builds a three-address instruction. $\|$ denotes concatenation of instruction sequences.

Production	**Semantic rules**
$S \to \mathbf{id} = E\ ;$	$S.code = E.code \,\|\| \mathbf{gen}(top.get(\mathbf{id}.lexeme) = E.addr)$
$E \to E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$
	$E.code = E_1.code \,\|\| E_2.code \,\|\| \mathbf{gen}(E.addr = E_1.addr + E_2.addr)$
$E \to E_1 * E_2$	$E.addr = \mathbf{new}\ Temp()$
	$E.code = E_1.code \,\|\| E_2.code \,\|\| \mathbf{gen}(E.addr = E_1.addr * E_2.addr)$
$E \to -\,E_1$	$E.addr = \mathbf{new}\ Temp()$
	$E.code = E_1.code \,\|\| \mathbf{gen}(E.addr = \texttt{minus}\ E_1.addr)$
$E \to (\,E_1\,)$	$E.addr = E_1.addr; \quad E.code = E_1.code$
$E \to \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme); \quad E.code = ''$

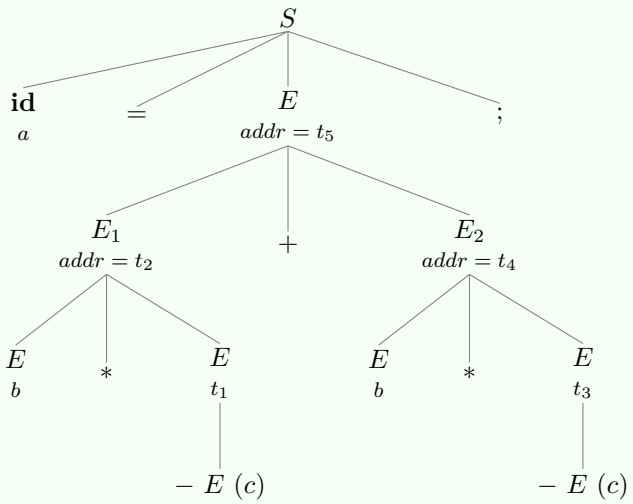
Table 6: SDD to produce three-address code for expressions and assignments (Dragon Book Fig. 6.19).

Remark

Here *top* is the current symbol table; $top.get(name)$ returns the symbol-table entry. The code of a node is the *concatenation* of its children's code followed by one new instruction, so the instructions are produced in postfix (evaluation) order. The *.code* attribute is a long string, so this SDD is inefficient in practice, which motivates the incremental approach below.

The grammar $E \to E_1 + E_2 \mid E_1 * E_2 \mid -E_1 \mid (E_1) \mid \mathbf{id}$ is ambiguous. Precedence ($-$ highest, then $*$, then $+$) and left-associativity are supplied either by parser-generator declarations or by rewriting the grammar with levels E, T, F. This does not change the semantic rules: they are written once per *operator*, whatever the grammar levels are called. The $*$ rule is the $+$ rule with the operator replaced; the rule for binary $-$ and for $/$ is identical again.

Worked example: $a = b * -c + b * -c$;



Concatenating the code in the order of evaluation gives:

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a  = t5
```

5.2 Incremental translation (SDT with `gen` emitting directly)

Building $E.code$ as a string copies code repeatedly. In an *incremental* scheme, $gen()$ **appends** the instruction to an output stream/array as soon as the action is executed. The code attribute is not stored. Since actions run in the correct order (postfix), the instructions come out in the right order.

Production	Translation scheme (actions in braces)
$S \to \mathbf{id} = E$;	{ $gen(top.get(\mathbf{id}.lexeme) = E.addr)$; }
$E \to E_1 + E_2$	{ $E.addr = \mathbf{new}\ Temp()$; $gen(E.addr = E_1.addr + E_2.addr)$; }
$\mid E_1 * E_2$	{ $E.addr = \mathbf{new}\ Temp()$; $gen(E.addr = E_1.addr * E_2.addr)$; }
$\mid -E_1$	{ $E.addr = \mathbf{new}\ Temp()$; $gen(E.addr =$ `minus` $E_1.addr)$; }
$\mid (E_1)$	{ $E.addr = E_1.addr$; }
$\mid \mathbf{id}$	{ $E.addr = top.get(\mathbf{id}.lexeme)$; }

Table 7: Incremental SDT for expressions (all actions are at the right end, so it is a postfix SDT, suitable for LR parsing).

Worked example (incremental SDT): `x = a * b + c * d ;`

An LR parser reduces from left to right, and every reduction that creates a temporary emits *one* instruction at that moment. Reductions $E \to \mathbf{id}$ only set $addr$ and emit nothing. The red numbers show the order in which instructions are emitted.

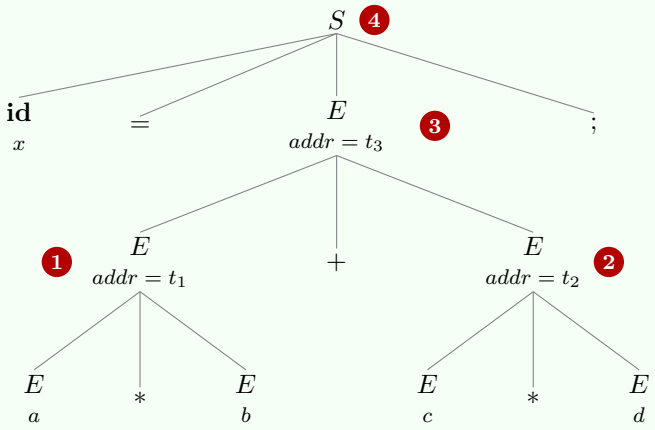


Figure 15: Parse tree of `x = a * b + c * d ;` with the order of emission.

#	Reduction	Instruction emitted
1	$E \to E_1 * E_2$ (`a * b`)	`t1 = a * b`
2	$E \to E_1 * E_2$ (`c * d`)	`t2 = c * d`
3	$E \to E_1 + E_2$	`t3 = t1 + t2`
4	$S \to \mathbf{id} = E\,;$	`x = t3`

Note that `c * d` is reduced only after the parser has seen the `;`: precedence decides that $*$ binds tighter, so the $+$ cannot be reduced earlier.

5.3 Addressing array elements

Let an array be stored in a contiguous block of memory with base address *base* and element width w.

One-dimensional array $A[low \ \ldots \ high]$:

$$\text{address of } A[i] \;=\; base + (i - low) \times w$$

If $low = 0$ this is simply $base + i \times w$.

Two-dimensional array (row-major) $A[n_1][n_2]$ with element width w:

$$\text{address of } A[i_1][i_2] \;=\; base + (i_1 \times n_2 + i_2) \times w$$

k dimensions $A[n_1]\cdots[n_k]$ (row-major, 0-based):

$$base + ((\cdots((i_1 n_2 + i_2)n_3 + i_3)\cdots)n_k + i_k) \times w$$

Equivalently, using the *widths of sub-arrays* $w_1 = n_2 \cdots n_k \cdot w,\ w_2 = n_3 \cdots n_k \cdot w, \ldots,\ w_k = w$:

$$base + i_1 \times w_1 + i_2 \times w_2 + \cdots + i_k \times w_k.$$

This form is the one used in the translation scheme.

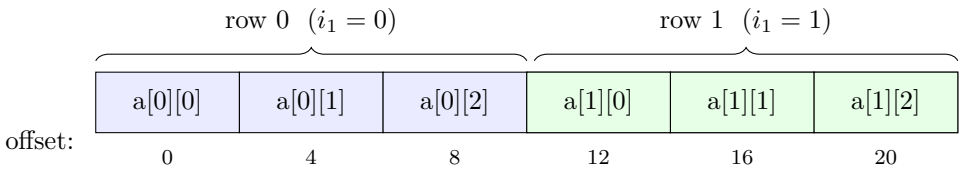


Figure 16: Row-major layout of `int a[2][3]` ($w = 4$). Offset of $a[i][j]$ is $i \times 12 + j \times 4$.

5.4 Translation of array references

Grammar (with nonterminal L for an array location):

$$S \to \mathbf{id} = E \ ; \ | \ L = E \ ;$$
$$E \to E_1 + E_2 \ | \ \mathbf{id} \ | \ L$$
$$L \to \mathbf{id}\,[\,E\,] \ | \ L_1\,[\,E\,]$$

Attributes of L: $L.array$ (symbol-table entry of the array), $L.type$ (type of the *sub-array* generated by L), $L.addr$ (a temporary accumulating the *offset*).

Production	**Semantic action**
$S \to \mathbf{id} = E$;	{ $gen(top.get(\mathbf{id}.lexeme) = E.addr)$; }
$S \to L = E$;	{ $gen(L.array.base\ [\ L.addr\ ] = E.addr)$; }
$E \to E_1 + E_2$	{ $E.addr = \mathbf{new}\ Temp()$; $gen(E.addr = E_1.addr + E_2.addr)$; }
$E \to \mathbf{id}$	{ $E.addr = top.get(\mathbf{id}.lexeme)$; }
$E \to L$	{ $E.addr = \mathbf{new}\ Temp()$; $gen(E.addr = L.array.base\ [\ L.addr\ ])$; }
$L \to \mathbf{id}\,[\,E\,]$	{ $L.array = top.get(\mathbf{id}.lexeme)$; $L.type = L.array.type.elem$; $L.addr = \mathbf{new}\ Temp()$; $gen(L.addr = E.addr * L.type.width)$; }
$L \to L_1\,[\,E\,]$	{ $L.array = L_1.array$; $L.type = L_1.type.elem$; $t = \mathbf{new}\ Temp()$; $L.addr = \mathbf{new}\ Temp()$; $gen(t = E.addr * L.type.width)$; $gen(L.addr = L_1.addr + t)$; }

Table 8: Semantic actions for array references (Dragon Book Fig. 6.22).

Worked example: $c + a[i][j]$ where `a` is `int a[2][3]`

Types: $a.type = array(2,\ array(3,\ integer))$, so the width of one row is $3 \times 4 = 12$ and of an element is 4.

1. $L \to \mathbf{id}[E]$ with $E = i$: $L.type = array(3, integer)$, width $12 \ \Rightarrow$ `t1 = i * 12`.
2. $L \to L_1[E]$ with $E = j$: $L.type = integer$, width $4 \ \Rightarrow$ `t2 = j * 4`, `t3 = t1 + t2`.
3. $E \to L$: `t4 = a[t3]`.
4. $E \to E_1 + E_2$: `t5 = c + t4`.

```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
t4 = a[t3]
t5 = c + t4
```

And for the assignment $x = a[i][j]$ the code is the first four instructions followed by `x = t4`.

Worked example: a three-dimensional array and a store

Let `int a[2][3][4]` (so $w = 4$). The widths of the sub-arrays are $w_3 = 4$ (one element), $w_2 = 4 \cdot 4 = 16$ (one innermost row of 4 elements) and $w_1 = 3 \cdot 4 \cdot 4 = 48$ (one 3×4 plane); the whole array occupies $2 \cdot 48 = 96$ bytes. The statement `x = a[i][j][k];` becomes

```
t1 = i * 48
t2 = j * 16
t3 = t1 + t2
t4 = k * 4
t5 = t3 + t4
t6 = a[t5]
x  = t6
```

Each extra subscript adds exactly two instructions (a multiplication and an addition), as in the rule for $L \to L_1[E]$. For a *store*, `a[i][j] = x;` with `int a[2][3]` uses the rule $S \to L = E$;, which ends with an indexed assignment instead of an indexed load:

```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
a[t3] = x
```

Index versus offset

In `t4 = a[t3]` the value in `t3` is a *byte offset* from the base of `a`, not an element index as in the C source. That is why every subscript is multiplied by the width of the thing it selects. (In the book's three-address code, `x = y[i]` means "the value at the memory location i units beyond y".)

6 Type Checking and Type Conversion

Type expressions represent the type of a language construct: basic types (`int`, `float`, `char`, `boolean`, `void`), $array(n, T)$, *record*, pointer, and function types $s \to t$.

Type checking comes in two forms: **synthesis** (the type of an expression is built from the types of its sub-expressions, e.g., for $E_1 + E_2$) and **inference** (the type of a construct is determined from the way it is used).

Example: three static checks

Static checking attaches a rule to a production and reports an error when the rule fails. With `int a[10]; int n; float f;`:

Source	Rule that fails	Message
`x = a[2.5];`	in $L \to \mathbf{id}[E]$ the index E must have type `int`	array index must be an integer
`y = a + n;`	$+$ is defined for numeric types only; a has type $array(10, \texttt{int})$	operand of $+$ is not a number
`if (f) x = 1;`	the condition in `if (`E`)` must have type `boolean`	`float` used as condition (C would accept it and test $f \neq 0$)

Each check is just one more semantic rule: it computes nothing, it only compares attribute

values and calls an error routine, which is another example of a rule with a side effect.

6.1 Type conversion (coercion)

For $E \to E_1 + E_2$ where operand types may differ (e.g., `int` and `float`), the compiler inserts conversions. Conversions that cannot lose information in principle (*widening*) may be inserted by the compiler automatically (*coercion*); the reverse direction (*narrowing*) needs an explicit cast written by the programmer. Figure 17 shows the usual hierarchy. In these notes we use only `int` < `float`.

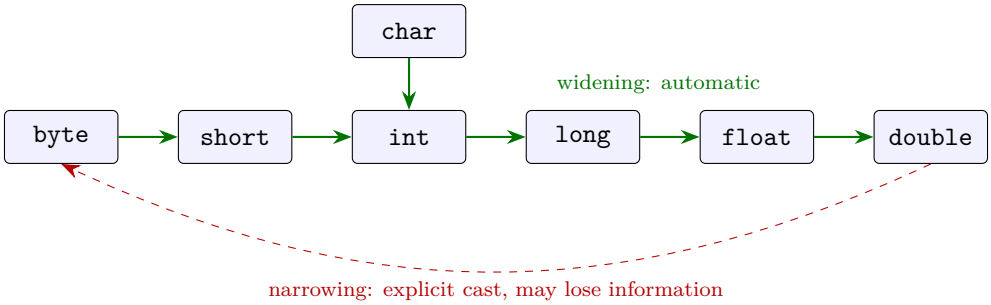


Figure 17: Widening (solid) and narrowing (dashed) conversions between primitive types, Java-style. Converting `long` to `float` is allowed implicitly although a few low-order digits may be rounded.

For $E \to E_1 + E_2$ (and likewise for $-$, $*$, $/$) the result has the *wider* of the two operand types,

$$E.type = \max(E_1.type,\ E_2.type),$$

and the helper $widen(a, t, w)$ converts address a of type t to type w:

```
Addr widen(Addr a, Type t, Type w) {
    if (t == w) return a;
    else if (t == integer && w == float) {
        temp = new Temp();
        gen(temp '=' '(float)' a);
        return temp;
    }
    else error;
}
```

Production	Semantic action
$E \to E_1 + E_2$	{ $E.type = \max(E_1.type, E_2.type)$; $a_1 = widen(E_1.addr, E_1.type, E.type)$; $a_2 = widen(E_2.addr, E_2.type, E.type)$; $E.addr =$ **new** $Temp()$; $gen(E.addr = a_1 + a_2)$; }

Example

Let `x` be `float`, and `i, j` be `int`. For $x = i * j + x$ we get:

```
t1 = i * j          // int * int -> int
t2 = (float) t1     // widen to float
t3 = t2 + x         // float + float
```

```
x  = t3
```

Worked example: widening on *both* sides, $x = i + j * x$

Let `x` be `float` and `i, j` be `int`. The code is emitted at reductions, so the product $j * x$ (reduced first) comes before the sum:

1. $E \to E_1 * E_2$ with $E_1 = j$ (`int`) and $E_2 = x$ (`float`): $E.type =$ `float`; $widen(j)$ emits `t1 = (float) j`; $widen(x)$ does nothing; then `t2 = t1 * x`.
2. $E \to E_1 + E_2$ with $E_1 = i$ (`int`) and E_2 of type `float`: $widen(i)$ emits `t3 = (float) i`; then `t4 = t3 + t2`.
3. $S \to \mathbf{id} = E$; emits `x = t4`.

```
t1 = (float) j
t2 = t1 * x
t3 = (float) i
t4 = t3 + t2
x  = t4
```

Note that i, which is written first, is converted *after* j: the order of the instructions follows the order of the reductions, not the order of the source text.

7 Control Flow: Boolean Expressions and Statements

In programming languages, boolean expressions are used for two purposes:

1. to **compute logical values** (e.g., `flag = a < b && c;`), and
2. as **conditional expressions** in `if`, `while`, and `for` (flow-of-control).

We use the grammar

$$B \to B \,\|\, B \;\mid\; B \;\&\&\; B \;\mid\; !B \;\mid\; (B) \;\mid\; E \;rel\; E \;\mid\; \mathbf{true} \;\mid\; \mathbf{false}$$

with $\|$ standing for the C operator `||`; `||` and `&&` are left-associative, `||` has lower precedence than `&&`, which is lower than `!`.

7.1 Short-circuit (jumping) code

In **short-circuit code**, the operators `&&`, `||` and `!` do not appear in the generated code. Instead a boolean expression is translated into jumps to labels: control *reaches* one position if the expression is true and another if it is false. In

```
if (x < 100 || x > 200 && x != y) x = 0;
```

the first test `x<100` being true makes the other tests unnecessary. The generated code is:

```
        if x < 100 goto L2
        goto L3
L3:     if x > 200 goto L4
        goto L1
L4:     if x != y goto L2
        goto L1
L2:     x = 0
L1:     ...
```

(Compare with *value* code, which would compute 0/1 into temporaries and use `&&` and `||` at run time.)

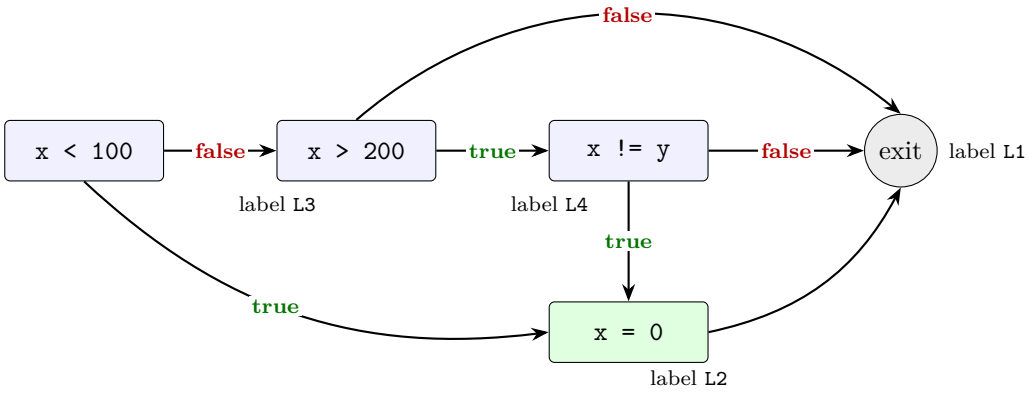


Figure 18: Control-flow view of `if (x < 100 || x > 200 && x != y) x = 0;`. Each test jumps on its own outcome; no boolean value is ever computed. A *true* arrow out of $x < 100$ skips the other tests.

Short-circuit evaluation is a semantic guarantee, not only an optimisation

In `if (p != NULL && p->x > 0) ...` the second operand must *not* be evaluated when p is null, otherwise the program dereferences a null pointer. Jumping code gives this behaviour for free: the test of `p->x` is only reached through the *true* exit of `p != NULL`. Value code that first computes both operands would crash.

Boolean values in assignments. A boolean expression is also used to compute a *value*, e.g. `flag = a < b && c;`. The jumping code is reused, and the two exits set the variable:

```
        if a < b goto L1
        goto L3
L1:     if c goto L2
        goto L3
L2:     flag = true
        goto L4
L3:     flag = false
L4:
```

7.2 Flow-of-control statements: SDD with inherited labels

Grammar for statements:

$$P \to S, \qquad S \to \textbf{assign} \mid \textbf{if}\,(B)\,S_1 \mid \textbf{if}\,(B)\,S_1\,\textbf{else}\,S_2 \mid \textbf{while}\,(B)\,S_1 \mid S_1\,S_2$$

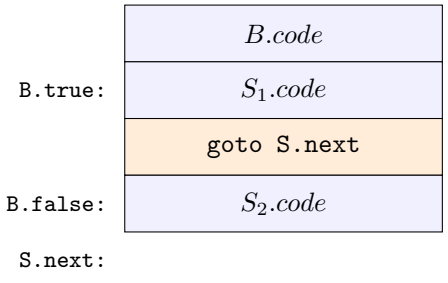
Inherited attributes (labels):

- $B.true$: the label to jump to if B is true; $B.false$: the label if B is false.
- $S.next$: the label of the instruction that follows the code for S.

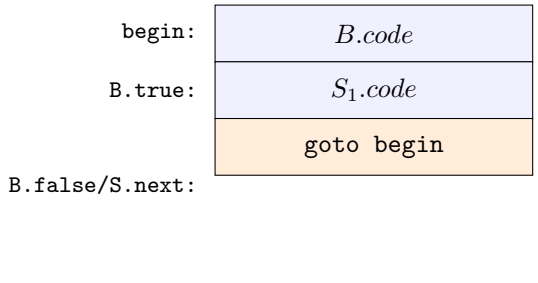
Synthesized attribute: $S.code$, $B.code$. The function $newlabel()$ creates a fresh label $L_1, L_2, \ldots$, and **label**(L) attaches label L to the next instruction.

Production	**Semantic rules**
$P \to S$	$S.next = newlabel();\quad P.code = S.code \parallel \textbf{label}(S.next)$
$S \to \textbf{assign}$	$S.code = \textbf{assign}.code$
$S \to \textbf{if}\,(B)\,S_1$	$B.true = newlabel();\quad B.false = S_1.next = S.next$ $S.code = B.code \parallel \textbf{label}(B.true) \parallel S_1.code$
$S \to \textbf{if}\,(B)\,S_1\ \textbf{else}\ S_2$	$B.true = newlabel();\quad B.false = newlabel();$ $S_1.next = S_2.next = S.next$ $S.code = B.code \parallel \textbf{label}(B.true) \parallel S_1.code \parallel \textbf{gen}(\textbf{goto}\ S.next) \parallel$ $\textbf{label}(B.false) \parallel S_2.code$
$S \to \textbf{while}\,(B)\,S_1$	$begin = newlabel();\quad B.true = newlabel();\quad B.false = S.next;$ $S_1.next = begin$ $S.code = \textbf{label}(begin) \parallel B.code \parallel \textbf{label}(B.true) \parallel S_1.code \parallel \textbf{gen}(\textbf{goto}\ begin)$
$S \to S_1\,S_2$	$S_1.next = newlabel();\quad S_2.next = S.next$ $S.code = S_1.code \parallel \textbf{label}(S_1.next) \parallel S_2.code$

Table 9: SDD for flow-of-control statements (Dragon Book Fig. 6.36). $S.next$, $B.true$, and $B.false$ are inherited.



(a) **if** (B) S_1 **else** S_2



(b) **while** (B) S_1

Figure 19: Layout of the generated code. $B.code$ contains jumps to $B.true$ or $B.false$; it "falls through" to neither, so the labels can be placed anywhere.

7.3 SDD for boolean expressions (jumping code)

Production	Semantic rules
$B \to B_1 \,\|\| B_2$	$B_1.true = B.true$; $B_1.false = newlabel()$; $B_2.true = B.true$; $B_2.false = B.false$ $B.code = B_1.code \,\|\|$ **label**$(B_1.false) \,\|\| B_2.code$
$B \to B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$; $B_1.false = B.false$; $B_2.true = B.true$; $B_2.false = B.false$ $B.code = B_1.code \,\|\|$ **label**$(B_1.true) \,\|\| B_2.code$
$B \to !\, B_1$	$B_1.true = B.false$; $B_1.false = B.true$; $B.code = B_1.code$
$B \to E_1\ rel\ E_2$	$B.code = E_1.code \,\|\| E_2.code \,\|\|$ **gen**(**if** $E_1.addr\ rel.op\ E_2.addr$ **goto** $B.true$) $\|\|$ **gen**(**goto** $B.false$)
$B \to$ **true**	$B.code =$ **gen**(**goto** $B.true$)
$B \to$ **false**	$B.code =$ **gen**(**goto** $B.false$)

Table 10: SDD to generate jumping code for boolean expressions (Dragon Book Fig. 6.37).

How to remember the `||` and `&&` rules

- $B_1 \,||\, B_2$: if B_1 is true we are done (go to $B.true$); if false, *test* B_2, so $B_1.false$ is a new label placed in front of $B_2.code$.
- $B_1 \;\&\&\; B_2$: if B_1 is false we are done (go to $B.false$); if true, *test* B_2, so $B_1.true$ is a new label in front of $B_2.code$.
- $!\, B_1$: simply *swap* the true and false labels.

Worked example: `if (x < 100 || x > 200 && x != y) x = 0;`

Step 1. $P \to S$: $S.next = L_1$.
Step 2. $S \to$ **if**$(B)S_1$: $B.true = L_2$, $B.false = L_1$, $S_1.next = L_1$.
Step 3. $B = B_1 \,||\, B_2$, where $B_1 \equiv x < 100$ and $B_2 \equiv (x > 200 \;\&\&\; x \neq y)$:

- B_1: true $= L_2$, false $= L_3$ (new).
- B_2: true $= L_2$, false $= L_1$.

Step 4. $B_2 = B_3 \;\&\&\; B_4$ with $B_3 \equiv x > 200$, $B_4 \equiv x \neq y$:

- B_3: true $= L_4$ (new), false $= L_1$.
- B_4: true $= L_2$, false $= L_1$.

Step 5. Assemble: $B_1.code \,||\, L_3$: $B_2.code$, with $B_2.code = B_3.code \,||\, L_4$: $B_4.code$, followed by L_2: $S_1.code$ and finally L_1:

```
        if x < 100 goto L2
        goto L3
L3:     if x > 200 goto L4
        goto L1
L4:     if x != y goto L2
        goto L1
L2:     x = 0
L1:
```

Observe the redundant jumps (`goto L3` immediately followed by `L3:`) – a later optimisation phase or the *backpatching* method with fall-through can remove them.

Worked example: `while (a < b) { if (c < d) x = y + z; }`

```
L1:     if a < b goto L2        // begin / B.code
        goto L0                 // B.false = S.next
L2:     if c < d goto L3        // inner if: B.true = L3
        goto L1                 // inner S.next = begin = L1
L3:     t1 = y + z
        x = t1
        goto L1                 // goto begin
L0:                             // S.next
```

7.4 Avoiding redundant gotos (fall-through)

Instead of

```
        if x > 200 goto L4
        goto L1
L4:     ...
```

we can generate the shorter form `ifFalse x > 200 goto L1` followed by the code at `L4`: if the true label is the next instruction (denote it by the special label *fall*), only one jump is needed. The rule for $B \to E_1\ rel\ E_2$ becomes: let $test = E_1.addr\ rel.op\ E_2.addr$:

Condition	Generated jump code
$B.true \neq fall$ and $B.false \neq fall$	`if test goto B.true  goto B.false`
$B.true \neq fall$ and $B.false = fall$	`if test goto B.true`
$B.true = fall$ and $B.false \neq fall$	`ifFalse test goto B.false`
both $= fall$	(no instruction)

The statement and operator rules change too. Let *fall* mean "the next instruction in sequence". The new rules (Dragon Book Section 6.6.5) are:

Production	Semantic rules
$S \to \textbf{if}\,(B)\,S_1$	$B.true = fall$; $B.false = S_1.next = S.next$; $S.code = B.code \parallel S_1.code$
$B \to B_1 \parallel B_2$	$B_1.true = (B.true$ if $\neq fall$, else a new label); $B_1.false = fall$; $B_2.true = B.true$; $B_2.false = B.false$
	$B.code = B_1.code \parallel B_2.code$ if $B.true \neq fall$; otherwise $B_1.code \parallel B_2.code \parallel \textbf{label}(B_1.true)$
$B \to B_1\ \&\&\ B_2$	$B_1.true = fall$; $B_1.false = (B.false$ if $\neq fall$, else a new label); $B_2.true = B.true$; $B_2.false = B.false$
	$B.code = B_1.code \parallel B_2.code$ if $B.false \neq fall$; otherwise $B_1.code \parallel B_2.code \parallel \textbf{label}(B_1.false)$

The extra label in the "otherwise" case is needed because when B_1 is decisive the control must jump *past* B_2, to the end of $B.code$.

Worked example: the same statement with fall-through

For `if (x < 100 || x > 200 && x != y) x = 0;` start with $B.true = fall$ and $B.false = L_1$. For $B_1 \,||\, B_2$: since $B.true = fall$, $B_1.true = L_2$ (new) and $B_1.false = fall$; B_2 inherits

$B.true = fall$ and $B.false = L_1$. For $B_2 = B_3 \,\&\&\, B_4$: $B_3.true = B_4.true = fall$ and $B_3.false = B_4.false = L_1$. The relational rule then gives

```
        if x < 100 goto L2
        ifFalse x > 200 goto L1
        ifFalse x != y goto L1
L2:     x = 0
L1:
```

Only 4 instructions (3 tests and the assignment) instead of 7, and no useless `goto`s.

8 Backpatching

8.1 The problem and the idea

In the one-pass, label-based scheme above we needed *inherited* labels ($B.true$, $S.next$) because a jump may be generated *before* its target is known. Inherited attributes make one-pass **bottom-up** translation awkward. **Backpatching** avoids this:

- When a jump is generated, its *target is left blank*.
- Each such incomplete jump is put on a **list** of jumps that will eventually go to the same target.
- When the target address becomes known, the instructions on the list are *patched* (filled in).

Thus we only need *synthesized* attributes (lists of instruction indices), and the whole translation can be done in a single bottom-up pass.

Backpatching primitives

- *makelist*(i): create a new list containing only i, an index into the instruction array; return a pointer to it.
- *merge*(p_1, p_2): concatenate the lists pointed to by p_1 and p_2; return the merged list.
- *backpatch*(p, i): insert i as the target label for each instruction on the list p.
- *nextinstr*: global variable containing the index of the next instruction to be generated. *emit*$(\dots)$ appends an instruction and increments it.

8.2 Boolean expressions

Synthesized attributes: $B.truelist$ (instructions that jump when B is true) and $B.falselist$ (when false). A **marker nonterminal** $M \to \varepsilon$ records the index of the next instruction ($M.instr$).

Worked example: $x < 100 \,||\, x > 200 \,\&\&\, x \neq y$ (instructions start at index 100)

Parse structure: $B \Rightarrow B_1 \,||\, M_1\, B_2$, where $B_2 \Rightarrow B_3 \,\&\&\, M_2\, B_4$.

1. $B_1 \equiv x < 100$: emit `100: if x<100 goto _`, `101: goto _`
   $B_1.truelist = \{100\}$, $B_1.falselist = \{101\}$.
2. $M_1.instr = 102$.
3. $B_3 \equiv x > 200$: emit `102: if x>200 goto _`, `103: goto _`
   $B_3.truelist = \{102\}$, $B_3.falselist = \{103\}$.
4. $M_2.instr = 104$.

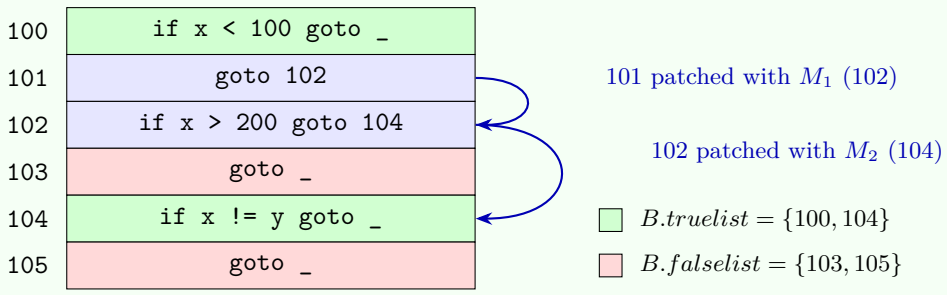
Production	Semantic action
$B \to B_1 \,\|\|\, M\ B_2$	{ $backpatch(B_1.falselist,\ M.instr)$; $B.truelist = merge(B_1.truelist,\ B_2.truelist)$; $B.falselist = B_2.falselist$; }
$B \to B_1\ \&\&\ M\ B_2$	{ $backpatch(B_1.truelist,\ M.instr)$; $B.truelist = B_2.truelist$; $B.falselist = merge(B_1.falselist,\ B_2.falselist)$; }
$B \to !\, B_1$	{ $B.truelist = B_1.falselist$; $B.falselist = B_1.truelist$; }
$B \to (\ B_1\ )$	{ $B.truelist = B_1.truelist$; $B.falselist = B_1.falselist$; }
$B \to E_1\ rel\ E_2$	{ $B.truelist = makelist(nextinstr)$; $B.falselist = makelist(nextinstr+1)$; $emit($`if` $E_1.addr\ rel.op\ E_2.addr$ `goto _`$)$; $emit($`goto _`$)$; }
$B \to$ **true**	{ $B.truelist = makelist(nextinstr)$; $emit($`goto _`$)$; }
$B \to$ **false**	{ $B.falselist = makelist(nextinstr)$; $emit($`goto _`$)$; }
$M \to \varepsilon$	{ $M.instr = nextinstr$; }

Table 11: Translation scheme for boolean expressions using backpatching (Dragon Book Fig. 6.43).

5. $B_4 \equiv x \neq y$: emit `104: if x!=y goto _`, `105: goto _`
   $B_4.truelist = \{104\}$, $B_4.falselist = \{105\}$.
6. Reduce $B_2 \to B_3\,\&\&\,M_2\,B_4$: $backpatch(\{102\}, 104)$ ⇒ instruction 102 becomes `if x>200 goto 104`.
   $B_2.truelist = \{104\}$, $B_2.falselist = merge(\{103\}, \{105\}) = \{103, 105\}$.
7. Reduce $B \to B_1 \,||\, M_1\,B_2$: $backpatch(\{101\}, 102)$ ⇒ instruction 101 becomes `goto 102`.
   $B.truelist = merge(\{100\}, \{104\}) = \{100, 104\}$, $B.falselist = \{103, 105\}$.

Result (still incomplete – the lists will be patched by the enclosing statement):

```
100:  if x < 100 goto _      <- on truelist
101:  goto 102
102:  if x > 200 goto 104
103:  goto _                 <- on falselist
104:  if x != y goto _       <- on truelist
105:  goto _                 <- on falselist
```



Blue jumps are already complete. The green and red jumps have a blank target; they are exactly the entries of the two lists that the parent will patch.

8.3 Flow-of-control statements

Synthesized attribute $S.nextlist$: list of jumps to the instruction that follows S. We use markers $M \to \varepsilon$ (records $nextinstr$) and $N \to \varepsilon$ (emits an incomplete `goto` and returns it on a list).

Grammar:

$$S \to \textbf{if}\,(B)\,S_1 \mid \textbf{if}\,(B)\,S_1\ \textbf{else}\ S_2 \mid \textbf{while}\,(B)\,S_1 \mid \{\,L\,\} \mid A\,;$$
$$L \to L_1\,S \mid S$$

Production	**Semantic action**
$S \to \textbf{if}\,(B)\ M\ S_1$	{ $backpatch(B.truelist,\ M.instr)$; $S.nextlist = merge(B.falselist,\ S_1.nextlist)$; }
$S \to$ **if** $(B)\ M_1\ S_1\ N$ **else** $M_2\ S_2$	{ $backpatch(B.truelist,\ M_1.instr)$; $backpatch(B.falselist,\ M_2.instr)$; $temp = merge(S_1.nextlist,\ N.nextlist)$; $S.nextlist = merge(temp,\ S_2.nextlist)$; }
$S \to$ **while** $M_1\,(B)\ M_2\ S_1$	{ $backpatch(S_1.nextlist,\ M_1.instr)$; $backpatch(B.truelist,\ M_2.instr)$; $S.nextlist = B.falselist$; $emit($`goto` $M_1.instr)$; }
$S \to \{\ L\ \}$	{ $S.nextlist = L.nextlist$; }
$S \to A\ ;$	{ $S.nextlist =$ **null**; }
$L \to L_1\ M\ S$	{ $backpatch(L_1.nextlist,\ M.instr)$; $L.nextlist = S.nextlist$; }
$L \to S$	{ $L.nextlist = S.nextlist$; }
$M \to \varepsilon$	{ $M.instr = nextinstr$; }
$N \to \varepsilon$	{ $N.nextlist = makelist(nextinstr)$; $emit($`goto _`$)$; }

Table 12: Translation of statements with backpatching (Dragon Book Fig. 6.46).

Why the markers?

M remembers a position: "this is where the code for S_1 (or for the test) begins". *N generates the jump over the else-part* and returns it for later patching. In the **while** rule the loop-back jump uses M_1, which was recorded *before* $B.code$ was generated.

Worked example: `while (a < b) { if (c < d) x = y + z; }`

Assume numbering starts at 100; $M_1.instr = 100$.

1. $B \equiv a < b$: `100: if a<b goto _`, `101: goto _`; $B.truelist = \{100\}$, $B.falselist = \{101\}$.
2. $M_2.instr = 102$.
3. Inner $B' \equiv c < d$: `102: if c<d goto _`, `103: goto _`; $B'.truelist = \{102\}$, $B'.falselist = \{103\}$.
4. $M.instr = 104$; assignment $x = y + z$: `104: t1 = y + z`, `105: x = t1`; $S_1.nextlist =$ **null**.
5. Inner **if**: $backpatch(\{102\}, 104)$; $S.nextlist = merge(\{103\}, \textbf{null}) = \{103\}$.
6. **while**: $backpatch(\{103\}, 100)$ (i.e., `103: goto 100`); $backpatch(\{100\}, 102)$ (i.e., `100: if a<b goto 102`); $S.nextlist = B.falselist = \{101\}$; emit `106: goto 100`.

```
100:  if a < b goto 102
101:  goto _              <- S.nextlist, patched by whoever follows
102:  if c < d goto 104
103:  goto 100
104:  t1 = y + z
```

```
105:  x = t1
106:  goto 100
107:  ...                  <- (101 will be backpatched with 107)
```

Worked example: `if (a < b) x = 1; else x = 2;`

```
100:  if a < b goto 102      // B.truelist = {100}, patched to M1 = 102
101:  goto 104               // B.falselist = {101}, patched to M2 = 104
102:  x = 1
103:  goto _                 // N: on S.nextlist
104:  x = 2
105:  ...                    // 103 is patched to 105 by the enclosing construct
```

8.4 Comparison: inherited labels vs. backpatching

	Label-based SDD (Fig. 6.36/6.37)	**Backpatching (Fig. 6.43/6.46)**
Attributes	Inherited labels (*true, false, next*)	Synthesized lists (*truelist, falselist, nextlist*)
Class of SDD	L-attributed	S-attributed (with markers M, N)
Best suited for	Top-down / recursive-descent (or AST traversal)	Bottom-up (LR) parsing, single pass
Code storage	Attribute *.code*, or incremental *gen*	Instruction array; patched later
Labels	Symbolic labels generated eagerly	Instruction indices known late

9 Switch Statements and Procedure Calls (Brief)

9.1 Switch statement

```
switch (E) {
    case V1: S1
    case V2: S2
    ...
    case V(n-1): S(n-1)
    default: Sn
}
```

Translation scheme: evaluate E into a temporary t, then a sequence of tests, followed by the code of each case.

```
        code to evaluate E into t
        goto test
L1:     code for S1
        goto next
L2:     code for S2
        goto next
 ...
Ln:     code for Sn
        goto next
test:   if t = V1 goto L1
        if t = V2 goto L2
```

```
        ...
        if t = V(n-1) goto L(n-1)
        goto Ln
next:
```

Putting the tests *after* the case bodies lets the code generator recognise the test sequence and implement it with a **jump table** (for dense values) or binary search. The `case` instruction can also be used: `case t V1 L1 ...  case t V(n-1) L(n-1)`; `case t t Ln` (the last entry is the default).

Break and fall-through

The scheme above puts `goto next` after every case body, which is the behaviour of Pascal-like `case` statements. In C and Java a body *falls through* into the next one unless it ends with `break`; to translate those, emit `goto next` only for `break`, and leave it out otherwise.

Worked example

```
switch (c) {
    case 1:  x = 10; break;
    case 2:  x = 20; break;
    default: x = 0;
}
```

(The selector is a variable, so no code is needed to evaluate it and $t = c$.)

```
        goto test
L1:     x = 10
        goto next
L2:     x = 20
        goto next
L3:     x = 0
        goto next
test:   if c = 1 goto L1
        if c = 2 goto L2
        goto L3
next:
```

Because the tests are collected at the end, the code generator can see that the values $1, 2, \ldots$ are dense and replace them by one indexed jump through a table $[L_1, L_2]$ after the range check $1 \le c \le 2$.

9.2 Procedure calls

For a call $id(E_1, E_2, \ldots, E_n)$:

```
        code to evaluate E1 into t1
        code to evaluate E2 into t2
        ...
        code to evaluate En into tn
        param t1
        param t2
        ...
        param tn
        call id, n          // or:  y = call id, n   if a value is returned
```

Grammar and actions (the parameter addresses are collected in a queue, and then emitted as `param`):

$S \to$ **call id** (*Elist*)	{ for each t in queue: *gen*(`param` t); *gen*(`call` **id**.*addr*, *size*(*queue*)); }
Elist $\to$ *Elist* , E	{ append $E.addr$ to the end of *queue*; }
Elist $\to E$	{ *queue* = [$E.addr$]; }

Worked example: $y = f(a+b,\ c*2)$

The arguments are evaluated first, left to right, then the `param` instructions are emitted, and finally the call:

```
t1 = a + b
t2 = c * 2
param t1
param t2
y = call f, 2
```

One global queue is not enough for nested calls

Take $f(a,\ g(b))$. After the first argument the queue of f is $[a]$. Parsing $g(b)$ then runs *Elist* $\to E$, which *resets* the same queue to $[b]$, destroying f's pending a. The fix is to keep the list as a synthesized attribute (*Elist.list*), or equivalently to use a *stack* of queues. The correct code is

```
param b
t1 = call g, 1
param a
param t1
t2 = call f, 2
```

10 A Complete Worked Example

Translating a small program fragment to three-address code

Source fragment (all variables are `int`; `a` is `int a[10]`, so $w = 4$):

```
i = 0;
while (i < n && a[i] != key)
    i = i + 1;
```

Using the label-based SDD. $P \to S$ gives $S.next = L_0$. For the sequence $S_1\,S_2$ (the assignment, then the loop) a label $L_1 = S_1.next$ is placed between them. For the loop, $begin = L_2$, $B.true = L_3$, $B.false = L_0$, and the body's $S_1.next = L_2$. For $B = B_1$ && B_2 a new label $L_4 = B_1.true$ is created, with $B_1.false = L_0$, $B_2.true = L_3$, $B_2.false = L_0$.

```
        i = 0
L1:
L2:     if i < n goto L4      // B1 (begin of while)
        goto L0               // B1 false -> exit loop
L4:     t1 = i * 4            // B2: a[i] != key
        t2 = a[t1]
        if t2 != key goto L3
        goto L0               // B false -> exit loop
L3:     t3 = i + 1            // loop body
        i = t3
```

```
        goto L2              // back to begin
L0:
```

(L1 and L2 label the same instruction; a later pass can merge them.)
Using backpatching (instruction numbering from 100). Here $M_1.instr = 101$ (start of loop), the marker of && gives 103, and $M_2.instr = 107$ (start of body).

```
100:  i = 0
101:  if i < n goto 103          // patched by the && marker
102:  goto _                     // on B.falselist = S.nextlist
103:  t1 = i * 4
104:  t2 = a[t1]
105:  if t2 != key goto 107      // truelist patched with M2 = 107
106:  goto _                     // on B.falselist = S.nextlist
107:  t3 = i + 1
108:  i = t3
109:  goto 101                   // emitted by the while rule (M1)
110:  ...                        // {102, 106} are backpatched with 110
```

Both of the incomplete jumps at 102 and 106 form *S.nextlist* of the loop; whoever follows the loop will patch them with the index of the next instruction, here 110.

11 Summary and Key Takeaways

- Intermediate code (syntax trees, DAGs, three-address code in quadruple/triple form) decouples front and back ends and enables machine-independent optimisation.
- A CFG gives syntax (is it a legal program, what is its structure) but not meaning. An **SDD** attaches *attributes* to grammar symbols and *semantic rules* to productions; **synthesized** attributes flow upward, **inherited** attributes flow downward or sideways. **S-attributed** $\subset$ **L-attributed**.
- An **SDT** embeds the actions in the grammar and fixes the order of evaluation. S-attributed SDDs $\rightarrow$ *postfix SDTs* (bottom-up). L-attributed SDDs can be implemented during top-down parsing, or, for LR grammars and with marker nonterminals, during bottom-up parsing.
- Expressions: *E.addr* holds the result; *gen* emits TAC; temporaries are created by *newTemp*. Array references use offsets computed from the widths of sub-arrays.
- Boolean expressions are translated into **jumping code**, using the inherited labels *true/false*. Statements use *next*.
- **Backpatching** replaces inherited labels by synthesized lists (*makelist, merge, backpatch*), allowing one-pass bottom-up generation with markers M and N.

Common mistakes (check your own answers against these)

- Reading a *value number* as the value of the expression. It is only the index of a record.
- Forgetting that a DAG shares a node only if *operator and both children* match; $b + a$ and $a + b$ are different unless the operands are put in canonical order.
- Using *B.true* where *S.next* is meant: *S.next* is where control goes *after* a statement, *B.true* is where it goes *into* the body.
- In a postfix SDT, putting an action in the middle of a body and expecting an LR parser to run it there; use a marker.
- Off-by-one errors in backpatching: *M.instr* is the index of the *next* instruction to be

emitted, not of the last one.
- Treating array subscripts as indices in the three-address code; they are scaled to byte offsets.
- Evaluating both operands of && / || in jumping code, which loses short-circuit behaviour.

12 Practice Problems

Q1. Translate $x = a * (b + c) - d/(b + c)$ into (a) a syntax tree and DAG, (b) three-address code, (c) quadruples and triples (and the order list that turns the triples into indirect triples).

Q2. Using the SDD of Table 6, annotate the parse tree and generate TAC for $a = -(b+c)*(b+c)$.

Q3. For `int a[5][4]` (int width = 4), write the TAC for $x = a[i][j] + a[j][i]$. Give the width used at each dimension.

Q4. Generate jumping code (with inherited labels) for `if (a < b && !(c < d)) x = 1; else x = 2;`

Q5. Show, step by step, the *truelist*/*falselist* and the final code (with backpatching, starting at 100) for $a < b \,||\, c < d \,\&\&\, e < f$.

Q6. Using backpatching, translate: `while (x < 10) { x = x + 1; if (x == 5) y = 0; }`

Q7. Classify as S-attributed, L-attributed, or neither: (a) $A \to B\,C$ with $C.i = B.s$ and $A.s = C.s$; (b) $A \to B\,C$ with $B.i = C.s$; (c) the SDD of Table 6.

Q8. Write the SDT for $E \to E_1 * E_2 \mid E_1 + E_2 \mid (E) \mid \textbf{id}$ that prints the *postfix* form, and show its output for `a + b * (c + d)`.

Q9. Build the array of value-number records for $a * b + a * b + c$ and give the three-address code obtained from it.

Q10. Translate `switch (n) { case 1: s = 1; break; case 5: s = 2; break; default: s = 0; }` using the scheme of Section 9.1.

Q11. Let `x` be `float` and `i, j` be `int`. Using the widening rules, write the code for $x = i + j * x$.

Q12. Generate the fall-through jumping code for `if (a < b || c < d) x = 1;` starting with $B.true = fall$.

Q13. Convert to SSA form: `x = 1; if (c) x = x + 1; y = x * 2;`

Q14. Convert to SSA form and draw the blocks: `k = 1; while (k < 10) k = k * 2; return k;`

Q15. For $x = (a + b) - (c * d)$, write the triples, then reorder the two sub-expressions. State what must be edited (a) with triples, (b) with indirect triples.

Q16. For each task say whether a CFG with its parse tree suffices or an SDD is needed: (a) accept `a = b + c;`, (b) reject the use of an undeclared variable, (c) compute the value of `3 * 5 + 4`, (d) check that both operands of + are numbers, (e) produce the parse tree of `a = b + c;`.

Answers to selected problems

Q1(b). The DAG shares $b + c$, so the TAC is

```
t1 = b + c
t2 = a * t1
t3 = d / t1
t4 = t2 - t3
x  = t4
```

Q3. Row width $= 4 \times 4 = 16$; element width $= 4$.

```
t1 = i * 16
t2 = j * 4
t3 = t1 + t2
t4 = a[t3]
t5 = j * 16
t6 = i * 4
t7 = t5 + t6
t8 = a[t7]
t9 = t4 + t8
x  = t9
```

Q4. $B.true = L_2$, $B.false = L_3$, $S.next = L_1$; for $B_1 \,\&\&\, B_2$ with $B_2 = !B_3$: $B_1.true = L_4$, $B_3.true = L_3$, $B_3.false = L_2$.

```
        if a < b goto L4
        goto L3
L4:     if c < d goto L3
        goto L2
L2:     x = 1
        goto L1
L3:     x = 2
L1:
```

Q7. (a) L-attributed ($C.i$ depends on the left sibling B) but not S-attributed. (b) Neither (inherited attribute of B depends on the right sibling C). (c) S-attributed (and hence L-attributed).

Q8. Postfix SDT: $E \to E_1 * E_2$ $\{print('*')\}$, $E \to E_1 + E_2$ $\{print('+')\}$, $E \to (E)$, $E \to$ **id** $\{print(\textbf{id}.lexeme)\}$ (with precedence/associativity declared or via an unambiguous grammar). Output for `a + b * (c + d)`: `a b c d + * +`.

Q2. Unary minus binds tighter than $*$, so the expression is $(-(b+c)) * (b+c)$. The SDD (no DAG) computes $b + c$ twice:

```
t1 = b + c
t2 = minus t1
t3 = b + c
t4 = t2 * t3
a  = t4
```

Q5. $B_1 \equiv a < b$ at 100–101, $M_1 = 102$, $B_3 \equiv c < d$ at 102–103, $M_2 = 104$, $B_4 \equiv e < f$ at 104–105. Reducing $B_3 \,\&\&\, M_2\, B_4$ patches $\{102\}$ with 104 and gives $truelist = \{104\}$, $falselist = \{103, 105\}$. Reducing $B_1 \,||\, M_1\, B_2$ patches $\{101\}$ with 102 and gives $truelist = \{100, 104\}$, $falselist = \{103, 105\}$.

```
100:  if a < b goto _     <- truelist
101:  goto 102
102:  if c < d goto 104
103:  goto _              <- falselist
104:  if e < f goto _     <- truelist
105:  goto _              <- falselist
```

Q6. $M_1 = 100$; test at 100–101; $M_2 = 102$; body: `102: t1 = x + 1`, `103: x = t1`; inner marker 104; inner test 104–105; inner marker 106; `106: y = 0`. The inner **if** has *nextlist* $= \{105\}$, which the **while** patches with $M_1 = 100$.

```
100:  if x < 10 goto 102
101:  goto _              <- S.nextlist (patched later)
102:  t1 = x + 1
103:  x = t1
104:  if x == 5 goto 106
105:  goto 100
106:  y = 0
107:  goto 100
108:  ...                 <- 101 is patched with 108
```

Q9. Records: 1 `id` a; 2 `id` b; 3 $(*, 1, 2)$; (second $a * b$ is found, value number 3); 4 $(+, 3, 3)$; 5 `id` c; 6 $(+, 4, 5)$. Code: `t1 = a * b`, `t2 = t1 + t1`, `t3 = t2 + c`.

Q10.

```
        goto test
L1:     s = 1
        goto next
L2:     s = 2
        goto next
L3:     s = 0
        goto next
test:   if n = 1 goto L1
        if n = 5 goto L2
        goto L3
next:
```

Here the values 1 and 5 are *sparse*, so the code generator would prefer a binary search or the comparison chain above to a jump table of five entries.

Q11.

```
t1 = (float) j
t2 = t1 * x
t3 = (float) i
t4 = t3 + t2
x  = t4
```

Q12. $B.true = fall$, $B.false = L_1$; for $B_1 \,||\, B_2$: $B_1.true = L_2$ (new), $B_1.false = fall$, $B_2.true = fall$, $B_2.false = L_1$.

```
        if a < b goto L2
        ifFalse c < d goto L1
L2:     x = 1
L1:
```

Q13. The test block branches to the then-block (which defines x_2) or directly to the join. At the join the two predecessors are the then-block and the test block, so

```
B0:  x1 = 1
     if c goto B1 else B2
B1:  x2 = x1 + 1              (then-block)
B2:  x3 = phi(x2, x1)         (join: pred. 1 = B1, pred. 2 = B0)
     y1 = x3 * 2
```

Note that a φ is needed even though there is no **else**: the "missing" branch is itself an edge into the join, carrying x_1.

Q14.

```
B0:  k1 = 1
B1:  k2 = phi(k1, k3)          (pred. 1 = B0, pred. 2 = B2)
     if k2 < 10 goto B2 else B3
B2:  k3 = k2 * 2               (back to B1)
B3:  return k2
```

Trace: $k_2 = 1, 2, 4, 8, 16$; the test fails for $k_2 = 16$ and the function returns 16.

Q15. Triples: $0 : (+, a, b)$, $1 : (*, c, d)$, $2 : (-, (0), (1))$, $3 : (=, x, (2))$. After evaluating $c * d$ first: (a) with triples the triples swap places, so triple 2 must become $(-, (1), (0))$ and every other reference to either of them must be rewritten; (b) with indirect triples the four triples stay exactly as they are and only the order list changes, from $(0), (1), (2), (3)$ to $(1), (0), (2), (3)$.

Q16. (a) CFG suffices (it is a syntax question). (b) SDD needed: it needs a symbol table and a rule that consults it. (c) SDD needed (an attribute *val*). (d) SDD needed (an attribute *type* and a rule comparing it). (e) CFG suffices: the parse tree is the output of syntax analysis.

References

1. A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2007. – Chapter 5 (Syntax-Directed Translation) and Chapter 6 (Intermediate-Code Generation).